



Performance Analysis

[Reattempt](#)[Download Certificate](#)

Detailed analysis of your performance in the AWS Certified Solutions Architect Associate - SAA C03 exam.

Answer Distribution

0

Correct

0

Incorrect

65

Unattempted

Domain Performance

Networking	0%
Serverless	0%
Databases	0%
Monitoring	0%
Cost Optimization	0%
Security	0%
Storage	0%
Compute	0%

Final Score

**0/65**

Passing score: 48

Time Taken

**130m 0s**

Total time: 130 minutes

Question Analysis

✓ Correct	0
✗ Incorrect	0
⊖ Unattempted	65
🔖 Reviewed	0

Question-wise Analysis

Expand All

Collapse All

Question 1

Not Attempted

Networking



A company operates business applications across three separate VPCs in the us-east-1 Region. These applications require inter-VPC communication. Additionally, the applications need to consistently transmit hundreds of gigabytes of data daily to a latency-sensitive application residing in a single on-premises data center. A solutions architect must design a network connectivity solution that maximizes cost-effectiveness. Which solution fulfills these requirements?

- ⊗ Configure three AWS Site-to-Site VPN connections from the data center to AWS. Establish connectivity by configuring one VPN connection for each VPC.
This option involves setting up multiple VPN connections, which can become complex and costly to manage for high data transfer volumes.
- ⊗ Launch a third-party virtual network appliance in each VPC. Establish an IPsec VPN tunnel between the data center and each virtual appliance.
Utilizing third-party virtual network appliances adds operational overhead and potential licensing costs, which may not be the most cost-effective approach.
- ⊗ Set up three AWS Direct Connect connections from the data center to a Direct Connect gateway in us-east-1. Establish connectivity by configuring each VPC to use one of the Direct Connect connections.
Establishing three Direct Connect connections is an expensive option. While it provides high bandwidth and low latency, it's not the most cost-effective solution for this scenario.
- ✓ Set up one AWS Direct Connect connection from the data center to AWS. Create a transit gateway, and attach each VPC to the transit gateway. Establish connectivity between the Direct Connect connection and the transit gateway.
This option uses a single Direct Connect for consistent, private connectivity between the data center and AWS. The transit gateway allows each VPC to share the Direct Connect while keeping the VPCs isolated. This provides a cost-effective architecture to meet the requirements.

Overall Explanation:

Option D provides the most cost-effective solution. A single AWS Direct Connect connection, combined with a transit gateway, facilitates high-bandwidth, low-latency connectivity between the on-premises data center and the AWS environment. The transit gateway acts as a central hub for connecting all VPCs, simplifying network management and eliminating complex peering relationships. This design minimizes costs by avoiding multiple Direct Connect connections while still meeting the data transfer and latency requirements.

[View Discussions](#)[Ask AI](#)

Question 2

Not Attempted

Serverless



An e-commerce company needs to build a distributed application for order processing, utilizing serverless functions and AWS services. The workflow includes tasks requiring manual approvals. A solutions architect must design an architecture that seamlessly integrates multiple AWS Lambda functions into responsive serverless applications while orchestrating data and services running on Amazon EC2 instances, containers, or on-premises servers. The solution must also support manual approvals. Which solution meets these requirements with the LEAST operational overhead?

☒ Use AWS Step Functions to build the application.

AWS Step Functions is a fully managed service designed for coordinating distributed application components and microservices using visual workflows. It enables combining multiple Lambda functions into serverless applications and orchestrating data and services across various compute environments, including EC2, containers, and on-premises servers. Step Functions also inherently supports manual approval processes within workflows, providing a solution with minimal operational overhead.

☐ Integrate all the application components in an AWS Glue job.

AWS Glue is primarily designed for ETL (Extract, Transform, Load) operations and data integration. While it can execute code and interact with other services, it's not optimized for orchestrating complex, long-running workflows with manual approval steps. Using Glue for this purpose would likely introduce significant operational overhead and complexity.

☐ Use Amazon Simple Queue Service (Amazon SQS) to build the application.

Amazon SQS is a message queuing service that facilitates asynchronous communication between application components. While SQS can be part of the solution, it does not provide workflow orchestration capabilities or direct support for manual approvals. Building the entire application logic with SQS would require significant custom development and increase operational overhead.

☐ Use AWS Lambda functions and Amazon EventBridge events to build the application.

While Lambda and EventBridge can be used to create event-driven architectures, building a complex, multi-step workflow with manual approvals using only these services would be challenging and result in a complex system with high operational overhead. It lacks the visual workflow and built-in state management capabilities of Step Functions.

Overall Explanation:

AWS Step Functions is the optimal solution because it is a fully managed workflow service that makes it easy to coordinate the components of distributed applications and microservices using visual workflows. It directly addresses all requirements: integrating Lambda functions, orchestrating services across different compute environments, and

supporting manual approvals, all with minimal operational overhead. Therefore, Option A is the correct answer.

[View Discussions](#)[Ask AI](#)

Question 3

Not Attempted

Databases



A company has launched an Amazon RDS for MySQL DB instance. Most connections to the database come from serverless applications. Application traffic to the database changes significantly at random intervals. During high demand, users report database connection rejection errors. Which solution resolves this issue with the LEAST operational overhead?

- ☒ Create a proxy in RDS Proxy. Configure the users' applications to use the DB instance through RDS Proxy.

RDS Proxy manages thousands of concurrent database connections, reducing connection errors. It offers features like connection pooling, read/write splitting, and retries with minimal operational overhead, avoiding instance migration or cache layer setup.

- ☐ Deploy Amazon ElastiCache for Memcached between the users' applications and the DB instance.

While ElastiCache can improve application performance, it does not directly address database connection limits and requires more operational overhead compared to RDS Proxy.

- ☐ Migrate the DB instance to a different instance class that has higher I/O capacity. Configure the users' applications to use the new DB instance.

Upgrading the instance class might improve performance, but it doesn't solve the underlying connection management issue and introduces operational overhead for migration.

- ☐ Configure Multi-AZ for the DB instance. Configure the users' applications to switch between the DB instances.

Multi-AZ provides high availability, but it doesn't directly address connection limits and requires application logic for failover, increasing complexity.

Overall Explanation:

RDS Proxy is designed to handle connection spikes from serverless applications. It pools and shares database connections, improving scalability and preventing connection

exhaustion. Configuring the applications to use RDS Proxy requires minimal changes and provides the least operational overhead compared to other options.

[View Discussions](#)[Ask AI](#)

Question 4

Not Attempted

Monitoring



A company recently deployed a new auditing system to centralize information about operating system versions, patching, and installed software for Amazon EC2 instances. A solutions architect must ensure all instances provisioned through EC2 Auto Scaling groups successfully send reports to the auditing system as soon as they are launched and terminated. Which solution achieves these goals MOST efficiently?

- ☐ Use a scheduled AWS Lambda function and run a script remotely on all EC2 instances to send data to the audit system.
Using a scheduled Lambda function is less efficient as it requires periodic execution and may not capture all instances launched and terminated within the interval.
- ☒ Use EC2 Auto Scaling lifecycle hooks to run a custom script to send data to the audit system when instances are launched and terminated.
EC2 Auto Scaling lifecycle hooks allow custom actions as instances launch and terminate. This ensures the auditing script executes at instance launch and termination, making it the most efficient solution.
- ☐ Use an EC2 Auto Scaling launch configuration to run a custom script through user data to send data to the audit system when instances are launched.
Using user data only addresses instance launch and doesn't automatically handle instance termination. Therefore, it is insufficient.
- ☐ Run a custom script on the instance operating system to send data to the audit system. Configure the script to be invoked by the EC2 Auto Scaling group when the instance starts and is terminated.
While this could work, directly invoking the script using Auto Scaling lifecycle hooks (option B) is a more direct and efficient approach.

Overall Explanation:

EC2 Auto Scaling lifecycle hooks provide the most efficient way to ensure all instances send reports to the auditing system upon launch and termination. Lifecycle hooks can delay instance termination until the script completes, ensuring data delivery. Launch configurations only handle launch, and Lambda is less efficient than lifecycle hooks. Therefore, option B is correct.

[View Discussions](#)[Ask AI](#)

Question 5

Not Attempted

Databases



A company is developing a real-time multiplayer game that uses UDP for communications between the client and servers in an Auto Scaling group. Spikes in demand are anticipated during the day, so the game server platform must adapt accordingly. Developers want to store gamer scores and other non-relational data in a database solution that will scale without intervention. Which solution should a solutions architect recommend?

- ☐ Use Amazon Route 53 for traffic distribution and Amazon Aurora Serverless for data storage.
Route 53 is for DNS, not application traffic distribution. Aurora Serverless is relational and not suitable for non-relational data.
- ☒ Use a Network Load Balancer for traffic distribution and Amazon DynamoDB on-demand for data storage.
Network Load Balancers can handle UDP traffic and DynamoDB on-demand offers scaling without intervention for non-relational data.
- ☐ Use a Network Load Balancer for traffic distribution and Amazon Aurora Global Database for data storage.
While Network Load Balancers can handle UDP traffic, Aurora Global Database is a relational database and not ideal for storing non-relational data.
- ☐ Use an Application Load Balancer for traffic distribution and Amazon DynamoDB global tables for data storage.
Application Load Balancers do not support UDP traffic. Although DynamoDB global tables can be used, the on-demand capacity mode might be more efficient for fluctuating workloads.

Overall Explanation:

The optimal solution is to use a Network Load Balancer (NLB) for distributing UDP traffic to the Auto Scaling group and Amazon DynamoDB in on-demand mode for storing non-relational data such as gamer scores. The NLB is designed for high-performance, low-latency traffic, which is crucial for real-time games. DynamoDB on-demand will automatically scale capacity based on the application's needs without any manual intervention. This combination addresses both the UDP requirement and the need for a scalable, non-relational database.

[View Discussions](#)[Ask AI](#)

Question 6

Not Attempted

Serverless



A company hosts a frontend application that uses an Amazon API Gateway API backend, integrated with AWS Lambda. When the API receives requests, the Lambda function loads many libraries and then connects to an Amazon RDS database, processes the data, and returns the data to the frontend application. The company wants to ensure that response latency is as low as possible for all its users with the fewest number of changes to the company's operations. Which solution will meet these requirements?

- ☐ Establish a direct connection between the frontend application and the database to make queries faster by bypassing the API.
Establishing a direct connection to the database from the frontend is not recommended due to security concerns and increased complexity in managing database access.
- ☒ Configure provisioned concurrency for the Lambda function that handles the requests.
Provisioned concurrency pre-initializes execution environments, ensuring they are ready to respond immediately to function invocations. This avoids cold starts and reduces latency caused by loading libraries for each invocation.
- ☐ Cache the results of the queries in Amazon S3 for faster retrieval of similar datasets.
Caching query results in S3 would require application changes and may not be effective if datasets are highly dynamic or unique.
- ☐ Increase the size of the database to increase the number of connections Lambda can establish at one time.
Increasing database size focuses on connection limits rather than the Lambda function's cold start or the efficiency of library loading. This doesn't directly address the latency issue of library loading.

Overall Explanation:

Provisioned concurrency for the Lambda function is the most effective solution for minimizing latency with the fewest operational changes. It eliminates cold starts by pre-initializing execution environments, ensuring the function is ready to respond immediately. This approach avoids the overhead of loading libraries on each invocation, thus improving response times.

[View Discussions](#)[Ask AI](#)

Question 7

Not Attempted

Networking



A company is hosting a static website on Amazon S3 and is using Amazon Route 53 for DNS. The website is experiencing increased demand from around the world. The company must decrease latency for users who access the website in the most cost-effective manner. Which solution meets these requirements?

- ⊗ Replicate the S3 bucket that contains the website to all AWS Regions. Add Route 53 geolocation routing entries.

Replicating the S3 bucket to all AWS Regions can be costly and complex, requiring data replication and synchronization management. It might not provide a significant latency improvement compared to CloudFront.

- ⊗ Provision accelerators in AWS Global Accelerator. Associate the supplied IP addresses with the S3 bucket. Edit the Route 53 entries to point to the IP addresses of the accelerators.

Provisioning accelerators in AWS Global Accelerator can be more expensive as it adds an extra layer of infrastructure and requires associating IP addresses with the S3 bucket. CloudFront already includes global edge locations and offers similar acceleration capabilities.

- ✓ Add an Amazon CloudFront distribution in front of the S3 bucket. Edit the Route 53 entries to point to the CloudFront distribution.

Adding an Amazon CloudFront distribution in front of the S3 bucket is the most cost-effective solution. CloudFront, a CDN, speeds up content delivery by distributing it across a global network of edge locations, reducing latency.

- ⊗ Enable S3 Transfer Acceleration on the bucket. Edit the Route 53 entries to point to the new endpoint.

Enabling S3 Transfer Acceleration helps improve upload speeds to the S3 bucket but may not significantly reduce latency for website visitors.

Overall Explanation:

Amazon CloudFront is a content delivery network (CDN) that speeds up the delivery of static and dynamic web content by distributing it across a global network of edge locations. It serves content to users from the nearest location, decreasing latency. It's also cost-effective because CloudFront charges are based on actual data transfer and requests. S3 Transfer Acceleration is for uploads, and replicating the bucket or using Global Accelerator are more expensive options. Therefore, using CloudFront is the most cost-effective way to decrease latency.

[View Discussions](#)[Ask AI](#)

Question 8

Not Attempted

Cost Optimization



A company is migrating its on-premises workload to the AWS Cloud. The company already uses several Amazon EC2 instances and Amazon RDS DB instances. The company requires a cost-effective and easily maintainable solution that automatically starts and stops the EC2 instances and DB instances outside of business hours. Which solution will meet these requirements?

- ☐ Scale the EC2 instances by using elastic resize. Scale the DB instances to zero outside of business hours.
DB instances cannot be scaled to zero, making this option infeasible.
- ☐ Explore AWS Marketplace for partner solutions that will automatically start and stop the EC2 instances and DB instances on a schedule.
While feasible, this might not be the most cost-effective and easily maintainable solution.
- ☐ Launch another EC2 instance. Configure a crontab schedule to run shell scripts that will start and stop the existing EC2 instances and DB instances on a schedule.
This adds unnecessary infrastructure and maintenance overhead.
- ☒ Create an AWS Lambda function that will start and stop the EC2 instances and DB instances. Configure Amazon EventBridge to invoke the Lambda function on a schedule.

This is the most efficient solution, leveraging serverless technology for cost and ease of maintenance. Lambda functions execute the start/stop commands, and EventBridge triggers the Lambda function on a schedule.

Overall Explanation:

The best solution is to use an AWS Lambda function triggered by Amazon EventBridge. This serverless approach minimizes cost and maintenance by automating the start/stop process without the need for dedicated servers. Lambda executes the start/stop scripts, while EventBridge schedules the Lambda function execution. Option A is incorrect because RDS instances cannot be scaled to zero. Option B might work but introduces potential costs and complexity. Option C requires managing another EC2 instance, increasing overhead.

[View Discussions](#)[Ask AI](#)

Question 9

Not Attempted

Databases



A company hosts a three-tier web application that includes a PostgreSQL database. The database stores metadata from documents stored in Amazon S3. The documents are generally written once but updated frequently. The company performs searches on the metadata to retrieve documents for a monthly report. This reporting process, using relational queries, takes several hours. The process must not interfere with ongoing document modifications or additions. A solutions architect needs to implement a solution to speed up the reporting process while minimizing changes to the application code. Which solution best meets these requirements?

- ☒ Set up a new Amazon DocumentDB (with MongoDB compatibility) cluster that includes a read replica. Scale the read replica to generate the reports.

DocumentDB requires significant code changes as it's a different database type than PostgreSQL. The application currently uses relational queries, which would need to be rewritten for DocumentDB's document-oriented model.

- ✔ Set up a new Amazon Aurora PostgreSQL DB cluster that includes an Aurora Replica. Issue queries to the Aurora Replica to generate the reports.

Aurora PostgreSQL provides PostgreSQL compatibility, minimizing code changes. An Aurora Replica separates the reporting workload from the main workload, preventing slowdowns during document updates/inserts. Aurora can also auto-scale read replicas to handle the reporting load effectively.

- ✘ Set up a new Amazon RDS for PostgreSQL Multi-AZ DB instance. Configure the reporting module to query the secondary RDS node so that the reporting module does not affect the primary node.

While Multi-AZ provides high availability, the secondary RDS node in a Multi-AZ setup is primarily for failover and is not designed for handling read workloads efficiently. Querying it for reporting can impact performance and is not the intended use case.

- ✘ Set up a new Amazon DynamoDB table to store the documents. Use a fixed write capacity to support new document entries. Automatically scale the read capacity to support the reports.

DynamoDB requires a significant rewrite of the data access code. The existing application uses relational queries on a PostgreSQL database, which are not directly compatible with DynamoDB's NoSQL model.

Overall Explanation:

Aurora PostgreSQL with a read replica is the optimal solution. It offers PostgreSQL compatibility, minimizing code changes, and allows offloading the reporting workload to the replica. This prevents any impact on the primary database handling document modifications and additions. DocumentDB and DynamoDB require extensive code changes, and RDS Multi-AZ isn't designed for read replica reporting.

 View Discussions

 Ask AI

Question 10

Not Attempted

Security



A company has a three-tier application on AWS that ingests sensor data from its users' devices. The traffic flows through a Network Load Balancer (NLB), then to Amazon EC2 instances for the web tier, and finally to EC2 instances for the application tier. The application tier makes calls to a database. What should a solutions architect do to improve the security of the data in transit?

- ✔ Configure a TLS listener. Deploy the server certificate on the NLB.

Configuring a TLS listener on the NLB and deploying a server certificate enables encryption of data in transit between the NLB and clients, as well as between the NLB and the backend EC2 instances. This protects against eavesdropping and tampering.

⊗ **Configure AWS Shield Advanced. Enable AWS WAF on the NLB.**

AWS Shield Advanced and AWS WAF protect against DDoS attacks and web application exploits like SQL injection or cross-site scripting. These are not directly related to encrypting data in transit.

⊗ **Change the load balancer to an Application Load Balancer (ALB). Enable AWS WAF on the ALB.**

While ALB also supports TLS and AWS WAF, switching to ALB is not the most direct way to improve security of data in transit. NLB already supports TLS. Also, WAF addresses different security concerns than data encryption.

⊗ **Encrypt the Amazon Elastic Block Store (Amazon EBS) volume on the EC2 instances by using AWS Key Management Service (AWS KMS).**

Encrypting EBS volumes protects data at rest. It does not protect data while it is being transmitted over the network.

Overall Explanation:

The best way to improve the security of data in transit is to use TLS encryption. NLB supports TLS termination, so the correct approach is to configure a TLS listener and deploy a server certificate on the NLB. This will encrypt the traffic between the users' devices and the NLB, as well as between the NLB and the backend EC2 instances. Options B, C and D are not related to protecting data in transit, addressing concerns such as DDoS attacks, web application vulnerabilities and data-at-rest encryption instead.

 [View Discussions](#)

 [Ask AI](#)

Question 11

Not Attempted

Cost Optimization



A company plans to migrate a commercial off-the-shelf (COTS) application from its on-premises data center to AWS. The application's software licensing model relies on sockets and cores with predictable capacity and uptime requirements. The company aims to utilize its existing licenses purchased earlier this year. Which Amazon EC2 pricing option is the MOST cost-effective for this scenario?

✓ **Dedicated Reserved Hosts**

Dedicated Reserved Hosts provide a billing discount compared to running On-Demand Dedicated Hosts and allow the use of existing socket/core-based licenses. The 'Reserved' aspect caters to predictable capacity and uptime, and 'Dedicated Host' allows bringing your own licenses (BYOL) for socket/core-based licensing.

✗ **Dedicated On-Demand Hosts**

Dedicated On-Demand Hosts allow using existing licenses, but are more expensive than Reserved Hosts because they don't offer a long-term commitment discount.

✗ **Dedicated Reserved Instances**

Dedicated Reserved Instances are cost-effective for predictable workloads but may not be compatible with all socket/core-based licensing models that require dedicated physical hardware. The licensing model of the COTS application necessitates visibility and control over physical cores, making this option less suitable.

✗ **Dedicated On-Demand Instances**

Dedicated On-Demand Instances do not allow you to bring your own licenses. Also, On-Demand pricing is generally more expensive compared to Reserved options when you have predictable capacity and uptime requirements.

Overall Explanation:

The MOST cost-effective option is Dedicated Reserved Hosts. They allow the company to leverage their existing licenses (BYOL), provide the necessary hardware isolation for socket/core-based licensing, and offer a significant discount due to the reserved capacity, catering to the predictable capacity and uptime requirements. Dedicated On-Demand Hosts, while allowing BYOL, lack the cost benefits of Reserved pricing. Dedicated Reserved Instances may not support the specific physical socket/core requirements of the application license, and Dedicated On-Demand Instances combine higher cost with the instance licensing constraints. Therefore, Dedicated Reserved Hosts best meet the combined needs of licensing and cost optimization.

 [View Discussions](#)

 [Ask AI](#)

Question 12

Not Attempted

[Storage](#)



A company runs an application on Amazon EC2 Linux instances across multiple Availability Zones. The application needs a storage layer that is highly available and Portable Operating System Interface (POSIX)-compliant. The storage layer must provide maximum data durability and must be shareable across the EC2 instances. The data in the storage layer will be accessed frequently for the first 30 days and will be accessed infrequently after that time. Which solution will meet these requirements MOST cost-effectively?

- ⊗ Use the Amazon S3 Standard storage class. Create an S3 Lifecycle policy to move infrequently accessed data to S3 Glacier.

S3 is object storage and not POSIX-compliant. Therefore, it will not work for applications requiring a file system interface.

- ⊗ Use the Amazon S3 Standard storage class. Create an S3 Lifecycle policy to move infrequently accessed data to S3 Standard-Infrequent Access (S3 Standard-IA).

S3 is object storage and not POSIX-compliant. Therefore, it will not work for applications requiring a file system interface.

- ✔ Use the Amazon Elastic File System (Amazon EFS) Standard storage class. Create a lifecycle management policy to move infrequently accessed data to EFS Standard-Infrequent Access (EFS Standard-IA).

EFS is POSIX-compliant, shareable, and supports lifecycle policies to move infrequently accessed data to a cheaper storage tier. EFS Standard offers maximum data durability and availability across multiple AZs.

- ⊗ Use the Amazon Elastic File System (Amazon EFS) One Zone storage class. Create a lifecycle management policy to move infrequently accessed data to EFS One Zone-Infrequent Access (EFS One Zone-IA).

While EFS One Zone is cheaper, it does not offer the same level of data durability as EFS Standard since the data is stored in a single AZ. This does not fulfill the 'maximum data durability' requirement.

Overall Explanation:

The requirements of POSIX compliance, high availability, maximum data durability, and shared storage across EC2 instances make Amazon EFS the appropriate choice. Using EFS Standard ensures durability and availability across multiple AZs. The lifecycle policy will optimize cost by moving infrequently accessed data to EFS Standard-IA after 30 days.

Options using S3 are incorrect because S3 is object storage and not POSIX-compliant. EFS One Zone doesn't provide Multi-AZ support and the required data durability.

[View Discussions](#)[Ask AI](#)

Question 13

Not Attempted

Networking



A solutions architect is designing a new VPC. The VPC includes two public subnets for the load balancer, two private subnets for web servers, and two private subnets for MySQL databases. The web servers only communicate over HTTPS. The architect has already configured a security group for the load balancer, allowing inbound traffic on port 443 from anywhere (0.0.0.0/0). Company policy mandates the principle of least privilege for all resources. What additional configuration should the solutions architect implement to meet these requirements?

- ☐ Create a security group for the web servers, allowing inbound port 443 from 0.0.0.0/0. Create a security group for the MySQL servers, allowing inbound port 3306 from the web servers security group.

This option violates the principle of least privilege by opening port 443 to the entire internet for web servers.

- ☐ Create a network ACL for the web servers, allowing inbound port 443 from 0.0.0.0/0. Create a network ACL for the MySQL servers, allowing inbound port 3306 from the web servers security group.

Network ACLs are stateless, and while this option attempts to limit MySQL access, it opens web server port 443 to the internet and uses Network ACLs where security groups are more appropriate.

- ☒ Create a security group for the web servers, allowing inbound port 443 from the load balancer security group. Create a security group for the MySQL servers, allowing inbound port 3306 from the web servers security group.

This option correctly implements least privilege. Web servers only accept traffic from the load balancer, and MySQL servers only accept traffic from the web servers.

- ☐ Create a network ACL for the web servers, allowing inbound port 443 from the load balancer. Create a network ACL for the MySQL servers, allowing inbound port 3306 from the web servers security group.

While this option attempts to limit access, it incorrectly uses network ACLs, which are stateless, for controlling access between instances. Security groups are more appropriate here.

Overall Explanation:

Option C adheres to the principle of least privilege. The web server security group allows only traffic from the load balancer security group on port 443, and the MySQL security group allows only traffic from the web server security group on port 3306. This minimizes the attack surface and provides granular control over network traffic. Security groups are stateful and operate at the instance level, making them the better choice compared to stateless network ACLs that operate at the subnet level.

[View Discussions](#)[Ask AI](#)

Question 14

Not Attempted

Databases



An e-commerce company's multi-tier application hosted on AWS experiences performance slowdowns due to frequent calls from the backend tier (running on EC2) to an Amazon RDS for MySQL database. These calls consistently return identical datasets. What is the MOST effective approach to improve the backend's performance and reduce the load on the RDS instance?

☐ Implement Amazon SNS to store the database calls.

Amazon SNS is a messaging service for pub/sub functionality and is not suitable for caching database query results.

☒ Implement Amazon ElastiCache to cache the large datasets.

Amazon ElastiCache provides in-memory caching for frequently accessed data, which can significantly reduce database load and improve response times, especially when identical datasets are repeatedly requested.

☐ Implement an RDS for MySQL read replica to cache database calls.

RDS Read Replicas are primarily used for read scaling and high availability, not for caching specific query results. They distribute read load, but do not provide the same performance benefits as a caching solution like ElastiCache for frequently accessed identical data.

☐ Implement Amazon Kinesis Data Firehose to stream the calls to the database.

Amazon Kinesis Data Firehose is used for streaming data to data lakes, data stores, and analytics services. It is not relevant for caching database query results.

Overall Explanation:

The optimal solution is to implement Amazon ElastiCache. Since the backend frequently requests identical datasets from the database, caching these datasets in ElastiCache (either Memcached or Redis) will significantly improve performance. ElastiCache provides a fast, in-memory data store that reduces the number of direct database queries, thereby minimizing latency and improving the overall responsiveness of the backend tier. Read replicas can help with scaling read operations, but are not optimized for caching specific query results like ElastiCache.

[View Discussions](#)[Ask AI](#)

Question 15

Not Attempted

Security



A new employee has joined a company as a deployment engineer. The deployment engineer will be using AWS CloudFormation templates to create multiple AWS resources. A solutions architect wants the deployment engineer to perform job activities while following the principle of least privilege. Which combination of actions should the solutions architect take to accomplish this goal? (Choose two.)

- ☐ Have the deployment engineer use AWS account root user credentials for performing AWS CloudFormation stack operations.
Using the root account violates the principle of least privilege. The root user has unrestricted access to all AWS services and resources.
- ☐ Create a new IAM user for the deployment engineer and add the IAM user to a group that has the PowerUsers IAM policy attached.
The PowerUsers IAM policy grants broad access, exceeding the principle of least privilege for a deployment engineer focused on CloudFormation.
- ☐ Create a new IAM user for the deployment engineer and add the IAM user to a group that has the AdministratorAccess IAM policy attached.
The AdministratorAccess IAM policy grants full access, violating the principle of least privilege. The deployment engineer only needs access to CloudFormation.
- ☒ Create a new IAM user for the deployment engineer and add the IAM user to a group that has an IAM policy that allows AWS CloudFormation actions only.
This option adheres to the principle of least privilege by granting only the necessary permissions for CloudFormation activities.

- ✓ Create an IAM role for the deployment engineer to explicitly define the permissions specific to the AWS CloudFormation stack and launch stacks using that IAM role.

Using an IAM role to define specific CloudFormation permissions and allowing the engineer to assume that role is a best practice for implementing least privilege.

Overall Explanation:

Options D and E are the correct choices. Option D involves creating a specific IAM policy that allows only CloudFormation actions, while option E involves using an IAM role with explicitly defined permissions for CloudFormation stack creation and management. Both these options ensure that the deployment engineer has the minimum required permissions to perform their job, adhering to the principle of least privilege. Options A, B and C grant excessive permissions and violate this principle.

 View Discussions

 Ask AI

Question 16

Not Attempted

Networking



A company is deploying a two-tier web application in a VPC. The web tier uses an Amazon EC2 Auto Scaling group with public subnets across multiple Availability Zones. The database tier consists of an Amazon RDS for MySQL DB instance in separate private subnets. The web tier needs to access the database to retrieve product information. The web application is not working as expected. It reports that it cannot connect to the database, although the database is confirmed to be up and running. All configurations for the network ACLs, security groups, and route tables are in their default states. What should a solutions architect recommend to resolve the connectivity issue?

- ✗ Add an explicit rule to the private subnet's network ACL to allow traffic from the web tier's EC2 instances.

The default network ACL allows all traffic, so the issue is likely not with the ACL.

- ✗ Add a route in the VPC route table to allow traffic between the web tier's EC2 instances and the database tier.

The default route table allows traffic within the VPC, so a new route is unnecessary.

- ⊗ Deploy the web tier's EC2 instances and the database tier's RDS instance into two separate VPCs, and configure VPC peering.

Deploying into two separate VPCs and configuring VPC peering is an overly complex solution to this problem.

- ✓ Add an inbound rule to the security group of the database tier's RDS instance to allow traffic from the web tier's security group.

By default, RDS instances block all inbound traffic. An inbound rule must be added to the RDS instance's security group allowing traffic from the web tier's security group.

Overall Explanation:

The default security group for the RDS instance blocks all inbound traffic. To allow the web tier to connect to the database, an inbound rule must be added to the RDS instance's security group to allow traffic from the web tier's security group. Network ACLs, route tables, and other configurations are at their default, so the security group on the RDS instance is the most likely cause of the connectivity issue.

 View Discussions

 Ask AI

Question 17

Not Attempted

Databases



A company has a large dataset for its online advertising business stored in an Amazon RDS for MySQL DB instance in a single Availability Zone. The company wants business reporting queries to run without impacting the write operations to the production DB instance. Which solution meets these requirements?

- ✓ Deploy RDS read replicas to process the business reporting queries.

RDS read replicas allow read-only copies of the production DB instance to be created. Queries to the read replica don't affect the source DB instance performance. This isolates reporting queries from production traffic and write operations.

- ⊗ Scale out the DB instance horizontally by placing it behind an Elastic Load Balancer.

Scaling out with an Elastic Load Balancer distributes traffic but doesn't isolate reporting queries from the production database, potentially impacting write operations.

- ⊗ Scale up the DB instance to a larger instance type to handle write operations and queries.

Scaling up the DB instance increases capacity but doesn't isolate reporting queries from the production database, potentially impacting write operations.

- ☐ Deploy the DB instance in multiple Availability Zones to process the business reporting queries.

Deploying the DB instance in multiple Availability Zones improves availability and provides failover capabilities, but doesn't isolate reporting queries from the production database, potentially impacting write operations.

Overall Explanation:

The best solution is to deploy RDS read replicas. Read replicas provide read-only copies of the production database, allowing reporting queries to run without impacting write operations to the production DB instance. This isolates reporting workload from production traffic.

 View Discussions

 Ask AI

Question 18

Not Attempted

Databases



A company maintains a searchable repository of items on its website. The data is stored in an Amazon RDS for MySQL database table that contains more than 10 million rows. The database has 2 TB of General Purpose SSD storage. There are millions of updates against this data every day through the company's website. The company has noticed that some insert operations are taking 10 seconds or longer. The company has determined that the database storage performance is the problem. Which solution addresses this performance issue?

- ☒ Change the storage type to Provisioned IOPS SSD.
Provisioned IOPS SSD is designed for I/O-intensive workloads requiring low latency and consistent throughput, directly addressing the storage performance bottleneck.
- ☐ Change the DB instance to a memory-optimized instance class.
Memory-optimized instances focus on memory capacity, which may not directly resolve the storage performance issue of slow insert operations.
- ☐ Change the DB instance to a burstable performance instance class.
Burstable performance instances are suitable for workloads with varying usage, but they may not provide consistent performance for heavy write workloads.

✘ Enable Multi-AZ RDS read replicas with MySQL native asynchronous replication.

Multi-AZ read replicas enhance high availability and read scaling, but they do not directly improve the insert performance on the primary database instance.

Overall Explanation:

The company is experiencing slow insert operations due to storage performance limitations. Provisioned IOPS SSD storage is specifically designed for high I/O workloads, allowing you to provision a specific level of IOPS to meet the performance requirements. This directly addresses the bottleneck and reduces latency for database operations, unlike the other options that focus on memory, burstable performance, or read scaling.

 View Discussions

 Ask AI

Question 19

Not Attempted

Databases



A company hosts a three-tier ecommerce application on a fleet of Amazon EC2 instances. The instances run in an Auto Scaling group behind an Application Load Balancer (ALB). All ecommerce data is stored in an Amazon RDS for MariaDB Multi-AZ DB instance. The company wants to optimize customer session management during transactions. The application must store session data durably. Which solutions will meet these requirements? (Choose two.)

✔ Turn on the sticky sessions feature (session affinity) on the ALB.

Enabling sticky sessions ensures that a user's requests are consistently routed to the same EC2 instance for the duration of their session. This improves performance by leveraging instance-local session data.

✘ Use an Amazon DynamoDB table to store customer session information.

DynamoDB provides durable storage for session data, allowing the application to persist session information reliably even if an EC2 instance fails or a user connects through a different instance.

✘ Deploy an Amazon Cognito user pool to manage user session information.

Amazon Cognito is primarily for user identity and authentication, not session data storage during transactions.

✔ Deploy an Amazon ElastiCache for Redis cluster to store customer session information.

ElastiCache for Redis provides a fast, in-memory data store for session data. It can improve application performance by reducing latency for session reads and writes. Redis also supports

replication for durability.

- ⊗ Use AWS Systems Manager Application Manager in the application to manage user session information.

AWS Systems Manager Application Manager is not designed for session management in applications.

Overall Explanation:

To optimize customer session management with durable storage, enabling sticky sessions on the ALB (Option A) ensures requests are routed to the same instance, and deploying an Amazon ElastiCache for Redis cluster (Option D) provides a fast, in-memory, and potentially durable (with replication) solution for storing session data. DynamoDB is a viable solution for durable session storage but ElastiCache for Redis provides faster access with options for persistence.

 View Discussions

 Ask AI

Question 20

Not Attempted

Databases



A company runs a three-tier stateless web application on Amazon EC2 instances within an Auto Scaling group. Scaling is dynamic, responding to events. The database tier uses Amazon RDS for PostgreSQL. The web application doesn't require local EC2 storage. The Recovery Point Objective (RPO) is 2 hours. Which backup strategy maximizes scalability and optimizes resource utilization?

- ⊗ Take snapshots of Amazon Elastic Block Store (Amazon EBS) volumes of the EC2 instances and database every 2 hours to meet the RPO.
- This option involves taking snapshots of EC2 instances, which is unnecessary for a stateless application that does not require local storage. This is also not a cost effective solution as it takes the snapshots every 2 hours regardless if there were changes made.
- ⊗ Configure a snapshot lifecycle policy to take Amazon Elastic Block Store (Amazon EBS) snapshots. Enable automated backups in Amazon RDS to meet the RPO.
- This option involves taking snapshots of EC2 instances, which is unnecessary for a stateless application that does not require local storage.

- ✓ Retain the latest Amazon Machine Images (AMIs) of the web and application tiers. Enable automated backups in Amazon RDS and use point-in-time recovery to meet the RPO.

This is the best approach. Since the application is stateless and doesn't use local storage, AMIs are sufficient for the web and application tiers. RDS automated backups with point-in-time recovery meet the database RPO.

- ✗ Take snapshots of Amazon Elastic Block Store (Amazon EBS) volumes of the EC2 instances every 2 hours. Enable automated backups in Amazon RDS and use point-in-time recovery to meet the RPO.

This option is not optimal because it suggests taking EBS snapshots for the EC2 instances, which is unnecessary given the stateless nature of the application and the lack of local storage requirements. Additionally, this is not a cost effective solution as it takes the snapshots every 2 hours regardless if there were changes made.

Overall Explanation:

The most efficient solution is to use AMIs for the web/application tiers (since they are stateless) and rely on RDS automated backups with point-in-time recovery for the database. This approach maximizes scalability and optimizes resource utilization because it doesn't involve unnecessary EBS snapshots of the EC2 instances. Since the web application does not require temporary local storage on the EC2 instances, snapshots of EBS volumes are not necessary. Retaining the latest Amazon Machine Images (AMIs) of the web and application tiers will provide only an image backup but automated backups in Amazon RDS and point-in-time recovery allows to backup the data which will meet the RPO.

 View Discussions

 Ask AI

Question 21

Not Attempted

Security



A company wants to deploy a new public web application on AWS. The application includes a web server tier that uses Amazon EC2 instances. The application also includes a database tier that uses an Amazon RDS for MySQL DB instance. The application must be secure and accessible for global customers that have dynamic IP addresses. How should a solutions architect configure the security groups to meet these requirements?

- ✓ Configure the security group for the web servers to allow inbound traffic on port 443 from 0.0.0.0/0. Configure the security group for the DB instance to allow inbound traffic on port 3306 from the security group of the web servers.

This is the correct answer. It allows HTTPS access from any public IP address, meeting the requirement for global customer access. The database security group only allows inbound port 3306 from the web server security group, properly restricting access to only the resources that need it.

- ✗ Configure the security group for the web servers to allow inbound traffic on port 443 from the IP addresses of the customers. Configure the security group for the DB instance to allow inbound traffic on port 3306 from the security group of the web servers.

This is incorrect because the IP addresses of the customers are dynamic and cannot be known in advance.

- ✗ Configure the security group for the web servers to allow inbound traffic on port 443 from the IP addresses of the customers. Configure the security group for the DB instance to allow inbound traffic on port 3306 from the IP addresses of the customers.

This is incorrect because the IP addresses of the customers are dynamic and cannot be known in advance. Also directly exposing the database to customer IPs is insecure.

- ✗ Configure the security group for the web servers to allow inbound traffic on port 443 from 0.0.0.0/0. Configure the security group for the DB instance to allow inbound traffic on port 3306 from 0.0.0.0/0.

This is incorrect because it opens the database to the entire internet, which is a security risk.

Overall Explanation:

The correct approach is to allow inbound HTTPS traffic (port 443) to the web servers from anywhere (0.0.0.0/0) since the customers have dynamic IPs. Then, the database security group should only allow inbound MySQL traffic (port 3306) from the web server's security group. This restricts access to the database to only the web servers, enhancing security.

 View Discussions

 Ask AI

Question 22

Not Attempted

Security



A payment processing company records all voice communication with its customers and stores the audio files in an Amazon S3 bucket. The company needs to capture the text from the audio

files. The company must remove from the text any personally identifiable information (PII) that belongs to customers. Which of the following solutions should a solutions architect implement to meet these requirements?

- ☐ Process the audio files by using Amazon Kinesis Video Streams. Use an AWS Lambda function to scan for known PII patterns.

Kinesis Video Streams is designed for real-time video ingestion and processing. While a Lambda function could be used, it's less efficient and doesn't natively offer PII redaction compared to other services.

- ☐ When an audio file is uploaded to the S3 bucket, invoke an AWS Lambda function to start an Amazon Textract task to analyze the call recordings.

Amazon Textract is designed for extracting text from documents and images, not primarily for audio files. It also doesn't offer built-in PII redaction for audio transcriptions.

- ☒ Configure an Amazon Transcribe transcription job with PII redaction turned on. When an audio file is uploaded to the S3 bucket, invoke an AWS Lambda function to start the transcription job. Store the output in a separate S3 bucket.

This is the most suitable solution. Amazon Transcribe offers PII redaction and can be triggered by an S3 event using Lambda. Storing the output in a separate bucket maintains the original audio and redacted text separately.

- ☐ Create an Amazon Connect contact flow that ingests the audio files with transcription turned on. Embed an AWS Lambda function to scan for known PII patterns. Use Amazon EventBridge to start the contact flow when an audio file is uploaded to the S3 bucket.

Amazon Connect is primarily for contact center solutions, not for batch processing audio files already stored in S3. While transcription can be integrated, using a separate Lambda function for PII scanning is less efficient than using Transcribe's built-in PII redaction.

Overall Explanation:

The optimal solution is to use Amazon Transcribe with PII redaction enabled. When a new audio file is uploaded to the S3 bucket, a Lambda function is triggered to start the transcription job. The transcribed text, with PII redacted, is then stored in a separate S3 bucket. This approach leverages Transcribe's built-in PII redaction capabilities, making it more efficient than manual PII scanning in a Lambda function.

 View Discussions

 Ask AI

Question 23

Not Attempted

Databases



A company is running a multi-tier ecommerce web application in the AWS Cloud. The application runs on Amazon EC2 instances with an Amazon RDS for MySQL Multi-AZ DB instance. Amazon RDS is configured with the latest generation DB instance with 2,000 GB of storage in a General Purpose SSD (gp3) Amazon Elastic Block Store (Amazon EBS) volume. The database performance affects the application during periods of high demand. A database administrator analyzes the logs in Amazon CloudWatch Logs and discovers that the application performance always degrades when the number of read and write IOPS is higher than 20,000. What should a solutions architect do to improve the application performance?

- ☐ Replace the volume with a magnetic volume.

Magnetic volumes have significantly lower IOPS capabilities than gp3, and would worsen the performance issue. Magnetic volumes support max 1000 IOPS.

- ☐ Increase the number of IOPS on the gp3 volume.

While increasing IOPS on gp3 is possible, io2 offers superior performance for high-demand workloads. GP3 scales up to 64,000 IOPS but io2 is designed for mission-critical workloads.

- ☒ Replace the volume with a Provisioned IOPS SSD (io2) volume.

Provisioned IOPS SSDs (io2) are specifically designed to deliver sustained high performance and low latency, making them ideal for database workloads requiring consistently high IOPS. They can handle more than 20,000 IOPS. RDS now supports io2 volumes.

- ☐ Replace the 2,000 GB gp3 volume with two 1,000 GB gp3 volumes.

Splitting the volume does not inherently increase the total IOPS capacity available to the database. While it might offer some concurrency benefits, it's unlikely to address the core performance bottleneck related to exceeding the IOPS limit. Two gp3 volume with 16000 each $2 \times 16000 = 32000$ IOPS. This might improve, but io2 is designed for the purpose.

Overall Explanation:

The best option is to replace the gp3 volume with a Provisioned IOPS SSD (io2) volume. io2 volumes are designed for high-performance database workloads that require consistently high IOPS and low latency. This will ensure the database can handle the demand during peak periods. Provisioning the necessary IOPS that exceeds 20,000 will handle the database workload during periods of high demand.

[View Discussions](#)[Ask AI](#)

Question 24

Not Attempted

Security



An IAM user made several configuration changes to AWS resources in their company's account during a production deployment last week. A solutions architect learned that a couple of security group rules are not configured as desired. The solutions architect wants to confirm which IAM user was responsible for making the changes. Which service should the solutions architect use to find the desired information?

☐ Amazon GuardDuty

Amazon GuardDuty is a threat detection service that continuously monitors for malicious activity and unauthorized behavior.

☐ Amazon Inspector

Amazon Inspector is an automated security assessment service that helps improve the security and compliance of applications deployed on AWS.

☒ AWS CloudTrail

AWS CloudTrail is a service that enables governance, compliance, operational auditing, and risk auditing of your AWS account. CloudTrail logs API calls made to AWS services, including the identity of the caller.

☐ AWS Config

AWS Config provides a detailed view of the configuration of AWS resources in your AWS account. It helps you assess, audit, and evaluate the configurations of your resources.

Overall Explanation:

AWS CloudTrail is the correct service to use to identify which IAM user made the configuration changes. CloudTrail logs API calls, including the user identity, timestamp, and service that was called.

[View Discussions](#)[Ask AI](#)

Question 25

Not Attempted

Security



A company has implemented a self-managed DNS service on AWS using EC2 instances in different AWS Regions. The endpoints are configured with a standard accelerator in AWS Global Accelerator. To protect the solution against DDoS attacks, what should a solutions architect do?

- ☒ Subscribe to AWS Shield Advanced. Add the accelerator as a resource to protect.

AWS Shield Advanced provides enhanced DDoS protection and can protect AWS Global Accelerator resources. Global Accelerator is the entry point exposed to the internet, making it the ideal target for DDoS protection.

- ☐ Subscribe to AWS Shield Advanced. Add the EC2 instances as resources to protect.

While you can add the accelerator as a resource to protect with AWS Shield Advanced, it's generally more effective to protect the individual resources (in this case, the EC2 instances) because AWS Shield Advanced will automatically protect resources associated with Global Accelerator.

- ☐ Create an AWS WAF web ACL that includes a rate-based rule. Associate the web ACL with the accelerator.

AWS WAF can provide some protection against web application attacks, but AWS Shield Advanced is more specifically designed to mitigate DDoS attacks at scale. While WAF can be used with Global Accelerator, it's not the primary solution for DDoS protection.

- ☐ Create an AWS WAF web ACL that includes a rate-based rule. Associate the web ACL with the EC2 instances.

AWS WAF can provide some protection against web application attacks, but AWS Shield Advanced is more specifically designed to mitigate DDoS attacks at scale. Protecting EC2 instances directly with WAF is less effective than protecting the entry point (Global Accelerator).

Overall Explanation:

AWS Shield Advanced is the appropriate service for protecting against DDoS attacks. Since the EC2 instances are behind Global Accelerator, protecting the accelerator is the most effective approach. AWS Shield Advanced can be subscribed to, and the accelerator can be added as a protected resource. WAF can complement the protection, but Shield Advanced is the primary service for DDoS mitigation.

 View Discussions

 Ask AI

Question 26

Not Attempted

Serverless



An e-commerce company stores sales records in an Amazon S3 bucket. Each object is up to 10 GB. A scheduled daily job aggregates and filters these records for analytics, taking up to an hour with constant, known CPU and memory usage. A solutions architect needs to minimize operational effort. Which solution meets these requirements?

- ⊗ Create an AWS Lambda function with an Amazon EventBridge trigger scheduled to run daily.

AWS Lambda has a 15-minute execution limit, which is insufficient for a job that can take up to an hour.

- ⊗ Create an AWS Lambda function integrated with an Amazon API Gateway HTTP API. Trigger the API with a daily Amazon EventBridge scheduled event.

AWS Lambda has a 15-minute execution limit, making this option unsuitable for the stated job duration.

- ✓ Create an Amazon Elastic Container Service (Amazon ECS) cluster with AWS Fargate launch type. Use an Amazon EventBridge scheduled event to launch an ECS task on the cluster.

This option uses Fargate, a serverless compute engine, minimizing operational overhead. It also supports the required job duration and simplifies scaling.

- ⊗ Create an Amazon Elastic Container Service (Amazon ECS) cluster with an Amazon EC2 launch type and an Auto Scaling group. Use an Amazon EventBridge scheduled event to launch an ECS task.

While functional, managing EC2 instances and Auto Scaling adds operational complexity compared to Fargate.

Overall Explanation:

Option C is the most suitable solution. Lambda functions (options A and B) are unsuitable due to their 15-minute time limit. Between C and D, Fargate (option C) is preferred for its serverless nature, minimizing operational overhead compared to managing EC2 instances with Auto Scaling (option D). Fargate handles scaling and infrastructure management, making it the ideal choice for minimizing operational effort while meeting the job's requirements.

 View Discussions

 Ask AI

Question 27

Not Attempted

Cost Optimization



A company needs to migrate 600 TB of sensitive data from its on-premises network-attached storage (NAS) to the AWS Cloud within 2 weeks. The data must be encrypted during transit, and the company has a 100 Mbps internet connection. Which solution offers the MOST cost-effective approach to meet these requirements?

- ☐ Use Amazon S3 multi-part upload functionality to transfer the files over HTTPS.

Transferring 600 TB of data over a 100 Mbps internet connection, even with multi-part upload, would take an impractically long time (estimated around 600 days). Therefore, this option is not feasible within the 2-week timeframe.

- ☐ Create a VPN connection between the on-premises NAS system and the nearest AWS Region. Transfer the data over the VPN connection.

Similar to option A, the 100 Mbps internet connection severely limits the transfer speed. Establishing a VPN doesn't solve the fundamental bandwidth limitation and would still result in an unacceptably long transfer time.

- ☒ Use the AWS Snow Family console to order several AWS Snowball Edge Storage Optimized devices. Use the devices to transfer the data to Amazon S3.

AWS Snowball Edge is designed for large-scale data transfers where network bandwidth is a constraint. It provides a secure and cost-effective way to physically ship data to AWS, bypassing the limitations of the internet connection. This option allows the company to meet the 2-week deadline.

- ☐ Set up a 10 Gbps AWS Direct Connect connection between the company location and the nearest AWS Region. Transfer the data over a VPN connection into the Region to store the data in Amazon S3.

While a 10 Gbps Direct Connect connection would significantly improve transfer speed, setting up a Direct Connect link takes time (potentially weeks), which exceeds the 2 week time limit and the costs involved are significant, making it less cost effective than Snowball Edge for a one-time data migration.

Overall Explanation:

The most cost-effective solution for transferring 600 TB of data within 2 weeks, given a limited internet bandwidth and security requirements, is to use AWS Snowball Edge. Options A and B are infeasible due to the slow internet connection. Option D, while potentially faster, is more expensive and has a longer setup time than the given deadline. Snowball edge is a purpose-built solution for this scenario.

[View Discussions](#)[Ask AI](#)

Question 28

Not Attempted

Security



A financial company hosts a web application on AWS. The application uses an Amazon API Gateway Regional API endpoint to provide users with current stock prices. The company's security team has observed an increase in the number of API requests and suspects potential HTTP flood attacks that could take the application offline. A solutions architect must design a solution to protect the application from such attacks with minimal operational overhead. Which solution meets these requirements?

- ☐ Create an Amazon CloudFront distribution in front of the API Gateway Regional API endpoint with a maximum TTL of 24 hours.
CloudFront with caching helps with static content, but won't effectively mitigate HTTP flood attacks directed at the API Gateway endpoint, as requests will still reach the origin after the cache expires.
- ☒ Create a Regional AWS WAF web ACL with a rate-based rule. Associate the web ACL with the API Gateway stage.
AWS WAF with a rate-based rule directly addresses HTTP flood attacks by limiting requests from specific IP addresses within a defined timeframe, providing effective protection with minimal operational overhead.
- ☐ Use Amazon CloudWatch metrics to monitor the Count metric and alert the security team when the predefined rate is reached.
Monitoring with CloudWatch will only alert the security team; it does not prevent or mitigate the attack. Manual intervention would be required, which increases operational overhead and response time.
- ☐ Create an Amazon CloudFront distribution with Lambda@Edge in front of the API Gateway Regional API endpoint. Create an AWS Lambda function to block requests from IP addresses that exceed the predefined rate.
While Lambda@Edge can block requests, this approach involves more configuration and management compared to a rate-based rule in AWS WAF, resulting in higher operational overhead.

Overall Explanation:

A Regional AWS WAF web ACL with a rate-based rule is the most suitable solution. AWS WAF is a managed web application firewall that can protect the API Gateway API from various attacks, including HTTP flood attacks. Rate-based rules allow configuring

thresholds to track request rates for a specified period and automatically block requests when the threshold is exceeded. Associating the web ACL with the API Gateway stage ensures that the protection is applied directly to the API. This approach minimizes operational overhead by leveraging managed services and automated protection mechanisms.

[View Discussions](#)[Ask AI](#)

Question 29

Not Attempted

Networking



An application runs on an Amazon EC2 instance in a VPC. The application processes logs that are stored in an Amazon S3 bucket. The EC2 instance needs to access the S3 bucket without connectivity to the internet. Which solution will provide private network connectivity to Amazon S3?

☒ Create a gateway VPC endpoint to the S3 bucket.

A gateway VPC endpoint allows you to connect to S3 bucket privately without additional cost. It provides private network connectivity between resources in a VPC and services like Amazon S3 without requiring an internet gateway, NAT gateway, or NAT instance. Traffic between the EC2 instance and the S3 bucket stays within the AWS private network.

☐ Stream the logs to Amazon CloudWatch Logs. Export the logs to the S3 bucket.

This option does not provide private network connectivity to S3. Log data might take time to become available for export. The requirement is direct EC2 to S3 connectivity.

☐ Create an instance profile on Amazon EC2 to allow S3 access.

Creating an instance profile just grants access but does not help EC2 connect to S3 privately. Instance profiles are used to assign IAM roles to an EC2 instance, but it is not related to network connectivity.

☐ Create an Amazon API Gateway API with a private link to access the S3 endpoint.

API Gateway is designed to forward requests to AWS Lambda, Amazon EC2, Elastic Load Balancing products, Amazon DynamoDB, Amazon Kinesis, or any publicly available HTTPS-based endpoint, but not directly to S3. It acts as a proxy receiving network from the outside.

Overall Explanation:

A gateway VPC endpoint for S3 allows the EC2 instance to access the S3 bucket without requiring internet connectivity. After you create the gateway endpoint, you can add it as a target in your route table for traffic destined from your VPC to Amazon S3. It establishes a private connection between your VPC and the S3 service without requiring internet connectivity. It ensures secure and direct communication between the EC2 instance and S3 with minimal operational complexity. VPC endpoint allows you to connect to AWS services using a private network instead of using the public Internet. With a gateway endpoint, you can access Amazon S3 from your VPC, without requiring an internet gateway or NAT device for your VPC, and with no additional cost.

[View Discussions](#)[Ask AI](#)

Question 30

Not Attempted

Serverless



A company has thousands of edge devices that collectively generate 1 TB of status alerts each day. Each alert is approximately 2 KB in size. A solutions architect needs to implement a solution to ingest and store the alerts for future analysis. The company wants a highly available solution. However, the company needs to minimize costs and does not want to manage additional infrastructure. Additionally, the company wants to keep 14 days of data available for immediate analysis and archive any data older than 14 days. What is the MOST operationally efficient solution that meets these requirements?

- ☒ Create an Amazon Kinesis Data Firehose delivery stream to ingest the alerts. Configure the Kinesis Data Firehose stream to deliver the alerts to an Amazon S3 bucket. Set up an S3 Lifecycle configuration to transition data to Amazon S3 Glacier after 14 days.

This is the most operationally efficient solution. Kinesis Data Firehose is a fully managed service for delivering real-time streaming data to destinations such as S3 and automates the data ingestion process. S3 Lifecycle configuration automatically transitions older data to Glacier, which is a cost effective for archiving.

- ☐ Launch Amazon EC2 instances across two Availability Zones and place them behind an Elastic Load Balancer to ingest the alerts. Create a script on the EC2 instances that will store the alerts in an Amazon S3 bucket. Set up an S3 Lifecycle configuration to transition data to Amazon S3 Glacier after 14 days.

This solution requires managing EC2 instances, load balancer and custom scripts. This increases the operational overhead and costs compared to fully managed services.

- ⊗ Create an Amazon Kinesis Data Firehose delivery stream to ingest the alerts. Configure the Kinesis Data Firehose stream to deliver the alerts to an Amazon OpenSearch Service (Amazon Elasticsearch Service) cluster. Set up the Amazon OpenSearch Service (Amazon Elasticsearch Service) cluster to take manual snapshots every day and delete data from the cluster that is older than 14 days.

While this solution can work, Amazon OpenSearch Service is primarily for search and analysis of large data sets in real-time which is not required. Furthermore, this option will be more expensive compared to S3 and the solution would require operational overhead in managing the snapshots and data retention.

- ⊗ Create an Amazon Simple Queue Service (Amazon SQS) standard queue to ingest the alerts, and set the message retention period to 14 days. Configure consumers to poll the SQS queue, check the age of the message, and analyze the message data as needed. If the message is 14 days old, the consumer should copy the message to an Amazon S3 bucket and delete the message from the SQS queue.

This solution is less efficient than using Kinesis Data Firehose and S3. It requires custom consumer logic to process messages, determine their age, and move them to S3, increasing operational complexity. SQS retention is limited to 14 days, so data older than that will be lost unless actively moved.

Overall Explanation:

The most operationally efficient solution is to use Amazon Kinesis Data Firehose to ingest the data into an Amazon S3 bucket, with an S3 Lifecycle policy to transition data to Amazon S3 Glacier after 14 days. This solution leverages fully managed services, minimizing operational overhead and costs while providing high availability and meeting all requirements.

[View Discussions](#)[Ask AI](#)

Question 31

Not Attempted

[Databases](#)

A company's application uses Amazon DynamoDB to store user profile data. The application's read traffic has significantly increased, leading to higher latency and increased costs. The company wants to reduce latency and minimize costs without modifying the application code. The company has a global user base, but most users are located in a single geographical region. Which solution should a solutions architect implement to meet these requirements with minimal changes?

✓ Enable DynamoDB Accelerator (DAX) cluster for the DynamoDB table.

DAX is an in-memory cache that can significantly reduce read latency and costs by caching frequently accessed data. It requires minimal application code changes.

✗ Implement DynamoDB Auto Scaling to increase the read capacity units (RCUs) of the DynamoDB table.

Auto Scaling adjusts RCUs dynamically based on traffic, but it doesn't directly reduce latency. It can also lead to higher costs if not configured properly.

✗ Migrate the DynamoDB table to a larger instance type.

DynamoDB is serverless and does not have instance types. Also, this option is not a solution for latency.

✗ Implement DynamoDB Global Tables and create a replica table in a region closer to the majority of users.

Global Tables are designed for multi-region applications with geographically distributed users. While it can reduce latency for users in the new region, it introduces complexity and might be overkill for a primarily single-region workload.

Overall Explanation:

DynamoDB Accelerator (DAX) is the best solution as it is designed to improve read performance and reduce costs for DynamoDB tables without requiring significant application code changes. It provides an in-memory cache that sits in front of DynamoDB, caching frequently accessed data and reducing the number of read requests that go to the DynamoDB service. Since most users are in a single region, DAX will offer the most immediate and cost-effective benefits without the complexity of Global Tables.

 View Discussions

 Ask AI

Question 32

Not Attempted

Databases



A company is migrating an existing application to the AWS Cloud to enhance its high availability and resilience. The current application is hosted in the company's on-premises data center. Recently, the application experienced data loss due to a database server crash caused by an unexpected power outage. The company requires a solution that eliminates single points

of failure and allows the application to scale based on user demand. Which solution meets these requirements?

- ☒ Deploy the application servers using Amazon EC2 instances in an Auto Scaling group across multiple Availability Zones. Use an Amazon RDS DB instance in a Multi-AZ configuration.

This option ensures high availability and resilience by distributing application servers across multiple Availability Zones. The Multi-AZ RDS configuration provides database redundancy, eliminating single points of failure and enabling automatic failover.

- ☐ Deploy the application servers using Amazon EC2 instances in an Auto Scaling group in a single Availability Zone. Deploy the database on an EC2 instance. Enable EC2 Auto Recovery.

This option has single points of failure. Deploying application servers in a single Availability Zone and relying on EC2 Auto Recovery does not provide sufficient redundancy for high availability.

- ☐ Deploy the application servers using Amazon EC2 instances in an Auto Scaling group across multiple Availability Zones. Use an Amazon RDS DB instance with a read replica in a single Availability Zone. Promote the read replica to replace the primary DB instance if the primary DB instance fails.

Having the read replica in a single AZ creates a single point of failure. Although the application servers are distributed, the database read replica being in a single Availability Zone limits the high availability of the solution.

- ☐ Deploy the application servers using Amazon EC2 instances in an Auto Scaling group across multiple Availability Zones. Deploy the primary and secondary database servers on EC2 instances across multiple Availability Zones. Use Amazon Elastic Block Store (Amazon EBS) Multi-Attach to create shared storage between the instances.

EBS Multi-Attach requires EC2 instances in the same AZ. While distributing the application servers is correct, using EBS Multi-Attach requires instances to be within the same Availability Zone, which does not provide high availability across Availability Zones.

Overall Explanation:

The optimal solution is to deploy application servers across multiple Availability Zones using an Auto Scaling group and configure the Amazon RDS DB instance in a Multi-AZ setup. This approach ensures both the application and the database are highly available and resilient, capable of automatically recovering from failures without any single point of failure.

[View Discussions](#)[Ask AI](#)

Question 33

Not Attempted

Databases



A company's application running on Amazon EC2 instances ingests and handles streaming data by sending it to Amazon Kinesis Data Streams (configured with default settings). Every other day, the application consumes this data and writes it to an Amazon S3 bucket for business intelligence (BI) processing. The company notices that Amazon S3 is not receiving all the data that the application sends to Kinesis Data Streams. What should a solutions architect do to resolve this issue?

- ☒ Update the Kinesis Data Streams default settings by modifying the data retention period.

The default retention period for Kinesis Data Streams is 24 hours. Since the data is consumed every other day (48 hours), increasing the retention period to at least 48 hours will ensure that all data is available for processing and writing to S3.

- ☐ Update the application to use the Kinesis Producer Library (KPL) to send the data to Kinesis Data Streams.

While KPL can improve the efficiency of data production, it doesn't address the issue of data loss due to retention period.

- ☐ Update the number of Kinesis shards to handle the throughput of the data that is sent to Kinesis Data Streams.

Increasing the number of shards will help with throughput but will not resolve the issue with data being lost due to the default 24-hour retention period. Increasing shards without increasing data retention means older records will still be dropped after 24 hours.

- ☐ Turn on S3 Versioning within the S3 bucket to preserve every version of every object that is ingested in the S3 bucket.

S3 Versioning helps preserve different versions of objects but doesn't address the initial issue of not receiving all data from Kinesis Data Streams.

Overall Explanation:

The core issue is that the default Kinesis Data Streams retention period of 24 hours is shorter than the data consumption frequency of every other day (48 hours). Therefore, updating the Kinesis Data Streams default settings by modifying the data retention period

to at least 48 hours will ensure that all data is available for processing and writing to Amazon S3.

[View Discussions](#)[Ask AI](#)

Question 34

Not Attempted

Serverless



A developer has an application using an AWS Lambda function to upload files to Amazon S3. The application requires the necessary permissions for this task. The developer already possesses an IAM user with valid credentials required for Amazon S3 access. What approach should a solutions architect implement to grant the Lambda function the required permissions to upload files to S3?

- ☐ Add the required IAM permissions to the resource policy of the Lambda function.
Adding permissions to the resource policy might work, but it's generally less secure and more complex to manage than using an IAM role. It doesn't follow best practices for least privilege.
- ☐ Create a signed request using the existing IAM user's credentials within the Lambda function.
Storing IAM user credentials directly within the Lambda function's code is a major security risk and should be avoided. Hardcoding credentials is never a good practice.
- ☐ Create a new IAM user and use the existing IAM user credentials in the Lambda function.
Creating another IAM user and using its credentials within the Lambda function presents the same security risks as option B. It does not solve the underlying problem of how to grant Lambda access to S3.
- ☒ Create an IAM execution role with the required permissions and attach the IAM role to the Lambda function.
This is the correct approach. An IAM execution role provides the Lambda function with temporary credentials to access other AWS services like S3. This method is secure, follows the principle of least privilege, and is the recommended best practice.

Overall Explanation:

The correct solution is to create an IAM execution role with the necessary permissions for the Lambda function to upload files to Amazon S3 and attach this role to the Lambda

function. This ensures that the Lambda function has the appropriate permissions without exposing sensitive credentials directly in the code. An IAM execution role is the recommended way to grant Lambda functions access to other AWS services, promoting secure and manageable access control.

[View Discussions](#)[Ask AI](#)

Question 35

Not Attempted

Serverless



A company has a serverless application where an AWS Lambda function is triggered upon new document uploads to an Amazon S3 bucket. This Lambda function processes these documents. Following a recent marketing campaign, the company observed that many documents were not processed. What should a solutions architect implement to enhance the application's architecture to ensure all documents are processed reliably?

- ☐ Set the Lambda function's runtime timeout value to 15 minutes.
Increasing the timeout might help if processing individual documents takes longer than the current timeout, but it doesn't address issues related to event loss during high traffic.
- ☐ Configure an S3 bucket replication policy. Stage the documents in the S3 bucket for later processing.
S3 replication copies objects between buckets. Staging documents for later processing doesn't solve the immediate issue of lost events and unprocessed documents during peak loads.
- ☐ Deploy an additional Lambda function. Load balance the processing of the documents across the two Lambda functions.
Adding another Lambda function might increase processing capacity, but without a mechanism to handle event bursts, events might still be missed. It doesn't inherently guarantee all documents will be processed if S3 events are dropped.
- ☒ Create an Amazon Simple Queue Service (Amazon SQS) queue. Send the requests to the queue. Configure the queue as an event source for Lambda.
Introducing an SQS queue decouples the S3 bucket from the Lambda function. SQS acts as a buffer, ensuring that events are not lost during traffic spikes. The Lambda function consumes messages from the queue, guaranteeing each document is processed even if the function is temporarily unavailable or overloaded.

Overall Explanation:

The most effective solution is to use Amazon SQS to decouple the S3 bucket and Lambda function. SQS provides a buffering mechanism to handle traffic spikes and ensure that all events are reliably delivered to the Lambda function for processing. This prevents event loss and ensures all uploaded documents are eventually processed. Options A, B and C do not effectively address the problem of event loss during high traffic periods.

[View Discussions](#)[Ask AI](#)

Question 36

Not Attempted

[Compute](#)

A solutions architect is designing the architecture for a software demonstration environment. The environment will run on Amazon EC2 instances in an Auto Scaling group behind an Application Load Balancer (ALB). The system will experience significant increases in traffic during working hours but is not required to operate on weekends. Which combination of actions should the solutions architect take to ensure that the system can scale to meet demand? (Choose two.)

- ☐ ☒ Use AWS Auto Scaling to adjust the ALB capacity based on request rate.
AWS Auto Scaling primarily manages the capacity of EC2 instances within an Auto Scaling group, not the ALB itself. While the ALB handles traffic distribution, Auto Scaling adjusts the number of instances behind it.
- ☐ ☒ Use AWS Auto Scaling to scale the capacity of the VPC internet gateway.
The VPC internet gateway's capacity is generally not a bottleneck and doesn't need to be scaled in response to application traffic changes.

- ☐ ☒ Launch the EC2 instances in multiple AWS Regions to distribute the load across Regions.
While multi-region deployment can enhance availability, it's not necessary for scaling to meet demand during working hours or for shutting down on weekends. It adds complexity and cost.

- ☒ ☐ Use a target tracking scaling policy to scale the Auto Scaling group based on instance CPU utilization.

A target tracking scaling policy allows the Auto Scaling group to automatically adjust the number of EC2 instances based on CPU utilization. This ensures that the application has sufficient resources to handle increased traffic during working hours.

- ✓ Use scheduled scaling to change the Auto Scaling group minimum, maximum, and desired capacity to zero for weekends. Revert to the default values at the start of the week.

Scheduled scaling allows you to define a scaling schedule to automatically adjust the Auto Scaling group's capacity. Setting the capacity to zero on weekends when the environment is not needed minimizes costs by terminating unused instances.

Overall Explanation:

To handle increased traffic during working hours, a target tracking scaling policy (Option D) is ideal for scaling the Auto Scaling group based on CPU utilization. To minimize costs on weekends when the environment is not required, scheduled scaling (Option E) can be used to set the Auto Scaling group's capacity to zero. Options A and B are incorrect because Auto Scaling doesn't directly adjust ALB capacity or scale the VPC internet gateway. Option C is an overkill for the given requirements.

 View Discussions

 Ask AI

Question 37

Not Attempted

Networking



A solutions architect is designing a two-tiered architecture that includes a public subnet and a database subnet. The web servers in the public subnet must be accessible from the internet on port 443. The Amazon RDS for MySQL DB instance in the database subnet must be accessible only to the web servers on port 3306. Which combination of steps should the solutions architect take to meet these requirements? (Choose two.)

- ✗ Create a network ACL for the public subnet. Add a rule to deny outbound traffic to 0.0.0.0/0 on port 3306.

Network ACLs are stateless and operate at the subnet level. While they can be used to deny traffic, security groups are better suited for instance-level control and easier to manage.

- ✗ Create a security group for the DB instance. Add a rule to allow traffic from the public subnet CIDR block on port 3306.

While this allows traffic on port 3306, it allows *any* resource within the public subnet to access the database, rather than specifically the web servers. This violates the principle of least privilege.

- ✔ Create a security group for the web servers in the public subnet. Add a rule to allow traffic from 0.0.0.0/0 on port 443.
This allows inbound traffic on port 443 (HTTPS) from anywhere on the internet to the web servers, fulfilling one of the requirements.
- ✔ Create a security group for the DB instance. Add a rule to allow traffic from the web servers' security group on port 3306.
This allows inbound traffic on port 3306 (MySQL) to the DB instance *only* from instances that are part of the web servers' security group, fulfilling the requirement of access limited to web servers.
- ✘ Create a security group for the DB instance. Add a rule to deny all traffic except traffic from the web servers' security group on port 3306.
Security groups operate on an 'allow' all, unless explicitly denied model. You cannot explicitly deny traffic. Anything not allowed, is implicitly denied.

Overall Explanation:

To allow web servers to be accessible on port 443 from the internet, you need to create a security group for the web servers allowing inbound traffic from 0.0.0.0/0 on port 443. To restrict database access to only the web servers, create a security group for the database, and allow inbound traffic on port 3306 only from the web servers' security group. This correctly restricts access to the database.

[View Discussions](#)[Ask AI](#)

Question 38

Not Attempted

Storage



A company is implementing a shared storage solution for a gaming application hosted in the AWS Cloud. The application requires Lustre clients to access data with minimal management overhead. Which of the following solutions meets these requirements?

- ✘ Create an AWS DataSync task that shares the data as a mountable file system. Mount the file system to the application server.
AWS DataSync is primarily used for data transfer and synchronization between on-premises storage and AWS storage services, not for providing a shared, mountable file system for applications to directly access data.

- ⊗ Create an AWS Storage Gateway file gateway. Create a file share that uses the required client protocol. Connect the application server to the file share.

AWS Storage Gateway file gateway provides file-based access to objects in S3, but it does not natively support the Lustre client protocol.

- ⊗ Create an Amazon Elastic File System (Amazon EFS) file system and configure it to support Lustre. Attach the file system to the origin server. Connect the application server to the file system.

Amazon EFS does not natively support the Lustre client protocol. It is designed for general-purpose file sharing and is not optimized for high-performance computing workloads that benefit from Lustre.

- ✔ Create an Amazon FSx for Lustre file system. Attach the file system to the origin server. Connect the application server to the file system.

Amazon FSx for Lustre is a fully managed file system designed for compute-intensive workloads and natively supports the Lustre client protocol, meeting both the performance and management requirements.

Overall Explanation:

Amazon FSx for Lustre is the correct answer. It's a fully managed service optimized for high-performance computing workloads and provides native support for Lustre clients. This eliminates the need for manual configuration and ensures the gaming application can efficiently access shared storage with minimal management overhead. EFS does not support Lustre, and DataSync and Storage Gateway are not designed for this use case.

 View Discussions

 Ask AI

Question 39

Not Attempted

Networking



A company receives UDP data from thousands of geographically dispersed remote devices. The data is processed immediately, and a message is sent back to the device if needed. No data is stored. The company requires a solution that minimizes data transmission latency and provides rapid failover to another AWS Region. Which solution best meets these requirements?

- ⊗ Configure an Amazon Route 53 failover routing policy. Create a Network Load Balancer (NLB) in each of the two Regions. Configure the NLB to invoke an AWS Lambda function to process the data.

This option utilizes Route 53 for failover, but Lambda functions are typically not suited for high-throughput UDP data processing due to potential latency and scaling limitations.

- ✔ Use AWS Global Accelerator. Create a Network Load Balancer (NLB) in each of the two Regions as an endpoint. Create an Amazon Elastic Container Service (Amazon ECS) cluster with the Fargate launch type. Create an ECS service on the cluster. Set the ECS service as the target for the NLB. Process the data in Amazon ECS.

This option leverages AWS Global Accelerator to minimize latency and provide rapid failover. The NLB supports UDP traffic, and ECS with Fargate offers a scalable and managed environment for data processing.

- ✘ Use AWS Global Accelerator. Create an Application Load Balancer (ALB) in each of the two Regions as an endpoint. Create an Amazon Elastic Container Service (Amazon ECS) cluster with the Fargate launch type. Create an ECS service on the cluster. Set the ECS service as the target for the ALB. Process the data in Amazon ECS.

While Global Accelerator provides low latency and failover, ALB does not natively support UDP traffic. The application requires UDP, making this option unsuitable.

- ✘ Configure an Amazon Route 53 failover routing policy. Create an Application Load Balancer (ALB) in each of the two Regions. Create an Amazon Elastic Container Service (Amazon ECS) cluster with the Fargate launch type. Create an ECS service on the cluster. Set the ECS service as the target for the ALB. Process the data in Amazon ECS.

This option uses Route 53 for failover, which can be slower than Global Accelerator for rapid failover. Furthermore, the ALB doesn't support UDP traffic, making this option inappropriate for the application's needs.

Overall Explanation:

The correct solution is to use AWS Global Accelerator with Network Load Balancers (NLBs) and Amazon ECS with Fargate. Global Accelerator minimizes latency and provides rapid failover across regions. NLBs handle the UDP traffic from the devices, and ECS with Fargate offers a scalable and managed environment to process this data. Route 53 failover and Application Load Balancers (ALBs) do not fit the requirements as well, with ALB being incompatible with the UDP protocol.

 View Discussions

 Ask AI

Question 40

Not Attempted

Storage



A solutions architect is tasked with migrating a Windows Internet Information Services (IIS) web application to AWS. The application currently relies on a file share hosted on the user's on-premises network-attached storage (NAS). The solutions architect proposes migrating the IIS web servers to Amazon EC2 instances in multiple Availability Zones connected to the storage solution, with an Elastic Load Balancer distributing traffic. Which service is the MOST resilient and durable replacement for the on-premises file share?

☐ Migrate the file share to Amazon RDS.

Amazon RDS is a relational database service and not suitable for replacing a file share.

☐ Migrate the file share to AWS Storage Gateway.

AWS Storage Gateway is a hybrid cloud storage service that connects on-premises applications to AWS storage services, but it doesn't directly replace a file share in the cloud.

☒ Migrate the file share to Amazon FSx for Windows File Server.

Amazon FSx for Windows File Server is a fully managed Windows file system service built on Windows Server, offering native SMB protocol support, high availability, durability, backup, and restore capabilities. It is designed for Windows-based workloads.

☐ Migrate the file share to Amazon Elastic File System (Amazon EFS).

Amazon EFS provides shared file storage primarily for Linux-based workloads and does not natively support Windows-based workloads.

Overall Explanation:

Amazon FSx for Windows File Server is the most appropriate solution for migrating a Windows-based file share to AWS. It offers native Windows compatibility, resilience, and durability. RDS is a database service, Storage Gateway is a hybrid solution, and EFS is primarily for Linux. Therefore, FSx is the correct choice.

 View Discussions

 Ask AI

Question 41

Not Attempted

Serverless



A company's application integrates with multiple software-as-a-service (SaaS) sources for data collection. The company uses Amazon EC2 instances to receive the data and to upload the data to an Amazon S3 bucket for analysis. The same EC2 instance that receives and uploads the data

also sends a notification to the user when an upload is complete. The company has noticed slow application performance and wants to improve the performance as much as possible. Which solution will meet these requirements with the LEAST operational overhead?

- ⊗ Create an Auto Scaling group so that EC2 instances can scale out. Configure an S3 event notification to send events to an Amazon Simple Notification Service (Amazon SNS) topic when the upload to the S3 bucket is complete.

While autoscaling improves performance, it doesn't address the SaaS integration as effectively as a managed service like AppFlow. It also doesn't necessarily have the *least* operational overhead.

- ✔ Create an Amazon AppFlow flow to transfer data between each SaaS source and the S3 bucket. Configure an S3 event notification to send events to an Amazon Simple Notification Service (Amazon SNS) topic when the upload to the S3 bucket is complete.

Amazon AppFlow is designed for SaaS integrations and offers a managed service for data transfer, reducing operational overhead. The S3 event notification handles user notifications, further simplifying the process.

- ⊗ Create an Amazon EventBridge (Amazon CloudWatch Events) rule for each SaaS source to send output data. Configure the S3 bucket as the rule's target. Create a second EventBridge (CloudWatch Events) rule to send events when the upload to the S3 bucket is complete. Configure an Amazon Simple Notification Service (Amazon SNS) topic as the second rule's target.

EventBridge can work, but managing multiple rules for each SaaS source increases operational overhead compared to AppFlow.

- ⊗ Create a Docker container to use instead of an EC2 instance. Host the containerized application on Amazon Elastic Container Service (Amazon ECS). Configure Amazon CloudWatch Container Insights to send events to an Amazon Simple Notification Service (Amazon SNS) topic when the upload to the S3 bucket is complete.

While containerization and ECS can improve resource utilization, the setup and management of ECS and CloudWatch Container Insights introduce more operational overhead than AppFlow.

Overall Explanation:

The solution with the least operational overhead is to use Amazon AppFlow to directly transfer data from SaaS sources to the S3 bucket. AppFlow is a fully managed service specifically designed for this purpose. Configuring an S3 event notification to trigger an SNS topic for upload completion notifications completes the solution without requiring complex infrastructure management. Therefore, option B is the correct answer.

[View Discussions](#)[Ask AI](#)

Question 42

Not Attempted

Storage



A company is deploying a new application on Amazon EC2 instances. The application writes data to Amazon Elastic Block Store (Amazon EBS) volumes. The company needs to ensure that all data that is written to the EBS volumes is encrypted at rest. Which solution will meet this requirement?

☐ Create an IAM role that specifies EBS encryption. Attach the role to the EC2 instances.
Attaching an IAM role to the EC2 instances does not automatically encrypt the EBS volumes.

☒ Create the EBS volumes as encrypted volumes. Attach the EBS volumes to the EC2 instances.

Creating encrypted EBS volumes and attaching them to EC2 instances ensures that all data written to the volumes is encrypted at rest. You can enable encryption for EBS volumes while creating them.

☐ Create an EC2 instance tag that has a key of Encrypt and a value of True. Tag all instances that require encryption at the EBS level.

Adding an EC2 instance tag does not ensure that the EBS volumes are encrypted.

☐ Create an AWS Key Management Service (AWS KMS) key policy that enforces EBS encryption in the account. Ensure that the key policy is active.

While a KMS key policy is required for encryption, solely relying on it without creating the EBS volumes as encrypted won't guarantee encryption at rest.

Overall Explanation:

The correct solution is to create the EBS volumes as encrypted volumes and attach them to the EC2 instances. When creating an EBS volume, you can specify whether to encrypt the volume. If encrypted, all data written to the volume is automatically encrypted at rest using AWS-managed keys or customer-managed keys (CMKs) stored in AWS KMS. The other options either do not meet the requirement of encrypting data at rest (A and C) or do so in a more complex or less efficient manner (D).

[View Discussions](#)[Ask AI](#)

Question 43

Not Attempted

Databases



A company hosts a web application with fluctuating usage. It experiences heavy loads at the beginning of each month, moderate loads at the start of each week, and variable loads throughout the week. The application currently runs on a web server and a MySQL database within their data center. The company intends to migrate the application to AWS and seeks a cost-effective database platform that requires minimal or no code modifications. Which of the following options best fits their needs?

**Amazon DynamoDB**

Amazon DynamoDB is a NoSQL database and would require significant application code changes to be compatible.

**Amazon RDS for MySQL**

Amazon RDS for MySQL provides a managed MySQL service, but it does not automatically scale up and down based on demand as effectively as Aurora Serverless. It may lead to higher costs due to over-provisioning.

**MySQL-compatible Amazon Aurora Serverless**

Aurora Serverless is a MySQL-compatible relational database engine that automatically scales compute and memory resources based on application usage. It is designed for infrequent, intermittent, or unpredictable workloads, making it a cost-effective choice. No upfront costs or commitments are required, and no database modifications are needed.

**MySQL deployed on Amazon EC2 in an Auto Scaling group**

Deploying MySQL on EC2 with Auto Scaling requires more operational overhead to manage the database instances and scaling. It is also less cost-effective than Aurora Serverless for highly variable workloads.

Overall Explanation:

Aurora Serverless is the most cost-effective solution for applications with sporadic and unpredictable usage patterns, as it automatically scales resources based on demand. Being MySQL-compatible, it requires minimal or no database modifications. DynamoDB is a NoSQL database, and RDS MySQL does not autoscale as well as Aurora Serverless. Deploying MySQL on EC2 with Auto Scaling requires significant operational overhead.

[View Discussions](#)[Ask AI](#)

Question 44

Not Attempted

Security



An image-hosting company stores its objects in Amazon S3 buckets. The company wants to avoid accidental exposure of the objects in the S3 buckets to the public. All S3 objects in the entire AWS account need to remain private. Which solution will meet these requirements?

- ☐ Use Amazon GuardDuty to monitor S3 bucket policies. Create an automatic remediation action rule that uses an AWS Lambda function to remediate any change that makes the objects public.

GuardDuty is a reactive approach and primarily focuses on detection rather than prevention. While it can identify public buckets, it doesn't inherently prevent them from becoming public in the first place.

- ☐ Use AWS Trusted Advisor to find publicly accessible S3 buckets. Configure email notifications in Trusted Advisor when a change is detected. Manually change the S3 bucket policy if it allows public access.

Trusted Advisor only provides checks and notifications. It relies on manual intervention, which is not proactive enough to ensure all objects remain private consistently.

- ☐ Use AWS Resource Access Manager to find publicly accessible S3 buckets. Use Amazon Simple Notification Service (Amazon SNS) to invoke an AWS Lambda function when a change is detected. Deploy a Lambda function that programmatically remediates the change.

AWS RAM is designed for sharing AWS resources across accounts, but it's not the primary tool for managing or restricting public access within a single account. This solution is also reactive.

- ☒ Use the S3 Block Public Access feature on the account level. Use AWS Organizations to create a service control policy (SCP) that prevents IAM users from changing the setting. Apply the SCP to the account.

This is the most effective solution. S3 Block Public Access on the account level prevents any S3 bucket from being made public, and using an SCP ensures that IAM users cannot override this setting.

Overall Explanation:

The best solution is to use S3 Block Public Access at the account level and then enforce this setting using an SCP via AWS Organizations. This ensures that all S3 buckets remain private and prevents any IAM user from accidentally or intentionally making them public. The other options are reactive and do not proactively prevent public access.

[View Discussions](#)[Ask AI](#)

Question 45

Not Attempted

Serverless



An e-commerce company's store, deployed on Amazon EC2 instances as a two-tier web application (web and database tiers), experiences increased user traffic and delays in sending timely marketing and order confirmation emails. The company aims to reduce email delivery issue resolution time and minimize operational overhead. Which solution best meets these requirements?

- ☐ Create a separate application tier using EC2 instances dedicated to email processing.
Creating a separate application tier adds complexity and operational overhead, which contradicts the requirement to minimize it.
- ☒ Configure the web instance to send email through Amazon Simple Email Service (Amazon SES).
Amazon SES is a cost-effective and scalable email service that simplifies email sending, reduces operational overhead, and minimizes time spent resolving complex email delivery issues.
- ☐ Configure the web instance to send email through Amazon Simple Notification Service (Amazon SNS).
Amazon SNS is not designed for sending marketing or order confirmation emails. It's primarily a messaging service for sending notifications to subscribed endpoints or clients.
- ☐ Create a separate application tier using EC2 instances dedicated to email processing. Place the instances in an Auto Scaling group.
This solution introduces unnecessary complexity and operational overhead compared to using Amazon SES.

Overall Explanation:

Amazon SES is the most suitable solution because it is specifically designed for sending high-volume emails efficiently and securely. It minimizes operational overhead and reduces the time spent resolving email delivery issues. Creating a separate application tier for email processing (options A and D) adds complexity and cost. Amazon SNS (option C) is not appropriate for transactional emails.

[View Discussions](#)[Ask AI](#)

Question 46

Not Attempted

Serverless



A company's business system generates hundreds of CSV reports daily and saves them to a network share. The company needs to store this data in the AWS Cloud for near-real-time analysis, minimizing administrative overhead. Which solution best meets these requirements?

- ☐ Use AWS DataSync to transfer the files to Amazon S3. Create a scheduled task that runs at the end of each day.

Using a scheduled task with DataSync doesn't provide near-real-time transfer, as the files will only be transferred at the end of the day. This increases latency.

- ☒ Create an Amazon S3 File Gateway. Update the business system to use a new network share from the S3 File Gateway.

S3 File Gateway caches data locally and automatically uploads it to S3, providing near-real-time access to data with minimal administrative overhead. It integrates seamlessly as a network share.

- ☐ Use AWS DataSync to transfer the files to Amazon S3. Create an application that uses the DataSync API in the automation workflow.

Creating an application to consume the DataSync API adds administrative overhead for development and maintenance. This is not the option with the least overhead.

- ☐ Deploy an AWS Transfer for SFTP endpoint. Create a script that checks for new files on the network share and uploads the new files by using SFTP.

Using SFTP requires a script to monitor the network share and upload new files, which increases administrative overhead. Also, it's more complex to manage compared to S3 File Gateway.

Overall Explanation:

The S3 File Gateway provides the least administrative overhead because it presents a simple network file share interface and automatically uploads files to S3 in near-real-time, without requiring scheduled tasks, scripts, or additional application development. DataSync requires either scheduling (not near-real time) or developing a separate application to trigger the DataSync API. SFTP also needs a custom script. Thus, S3 File Gateway is the correct answer.

[View Discussions](#)[Ask AI](#)

Question 47

Not Attempted

Cost Optimization



A company is storing petabytes of data in Amazon S3 Standard. The data is stored in multiple S3 buckets and is accessed with varying frequency. The company does not know access patterns for all the data. The company needs to implement a solution for each S3 bucket to optimize the cost of S3 usage. Which solution will meet these requirements with the MOST operational efficiency?

- ☒ Create an S3 Lifecycle configuration with a rule to transition the objects in the S3 bucket to S3 Intelligent-Tiering.
S3 Intelligent-Tiering automatically moves objects between access tiers based on changing access patterns, offering cost optimization with minimal operational overhead, especially when access patterns are unknown.
- ☐ Use the S3 storage class analysis tool to determine the correct tier for each object in the S3 bucket. Move each object to the identified storage tier.
While S3 Storage Class Analysis provides insights, manually moving objects based on its analysis is less operationally efficient than using S3 Intelligent-Tiering, especially across multiple buckets.
- ☐ Create an S3 Lifecycle configuration with a rule to transition the objects in the S3 bucket to S3 Glacier Instant Retrieval.
S3 Glacier Instant Retrieval is suitable for infrequently accessed data that needs immediate retrieval. It might not be the best choice if access patterns are unknown as some data might be accessed more frequently.
- ☐ Create an S3 Lifecycle configuration with a rule to transition the objects in the S3 bucket to S3 One Zone-Infrequent Access (S3 One Zone-IA).
S3 One Zone-IA is suitable for infrequently accessed data that can tolerate data loss in a single availability zone. Similar to Glacier Instant Retrieval, it might not be the best choice when access patterns are unpredictable.

Overall Explanation:

S3 Intelligent-Tiering is the most operationally efficient solution because it automatically manages data placement based on access patterns without requiring manual intervention. Since the company does not know the access patterns for the data, Intelligent-Tiering is

the ideal solution. The other options require manual analysis and configuration, which is less efficient.

[View Discussions](#)[Ask AI](#)

Question 48

Not Attempted

Databases



A rapidly growing global ecommerce company is hosting its web application on AWS. The web application includes static content and dynamic content. The website stores online transaction processing (OLTP) data in an Amazon RDS database. The website's users are experiencing slow page loads. Which combination of actions should a solutions architect take to resolve this issue? (Choose two.)

☐ Configure an Amazon Redshift cluster.

Amazon Redshift is a data warehousing service typically used for analytical processing (OLAP), not for improving the performance of an OLTP database or delivering web content.

☒ Set up an Amazon CloudFront distribution.

Amazon CloudFront can cache static content globally and improve latency for users accessing the website. It can also be used for dynamic content acceleration.

☐ Host the dynamic web content in Amazon S3.

Amazon S3 is object storage and not suitable for hosting dynamic web content that requires server-side processing.

☒ Create a read replica for the RDS DB instance.

Creating a read replica can offload read traffic from the primary database instance, reducing the load on the primary and improving the performance of read-heavy operations.

☐ Configure a Multi-AZ deployment for the RDS DB instance.

Configuring a Multi-AZ deployment for RDS improves high availability and provides failover capabilities, but may not directly resolve the slow page load issue. While it improves database resilience, it primarily addresses availability rather than performance.

Overall Explanation:

To resolve the slow page load issue, the solutions architect should set up an Amazon CloudFront distribution to cache static content and create a read replica for the RDS DB

instance to offload read traffic. Redshift is for OLAP, not OLTP. S3 cannot host dynamic content. Multi-AZ is for HA, not performance.

[View Discussions](#)[Ask AI](#)

Question 49

Not Attempted

Networking



A company runs its application on Amazon EC2 instances and AWS Lambda functions within a VPC that has public and private subnets. EC2 instances reside in a private subnet. Lambda functions require direct network access to the EC2 instances for proper application functionality. The application is expected to run for at least a year, with the number of Lambda functions increasing over time. The company aims to minimize costs for all application resources while maintaining low network latency between the services. Which solution fulfills these requirements?

- ☐ Purchase an EC2 Instance Savings Plan. Optimize the Lambda functions' duration, memory usage, and the number of invocations. Connect the Lambda functions to the private subnet containing the EC2 instances.

While EC2 Instance Savings Plans offer cost savings on EC2, they don't cover Lambda costs. This makes the solution incomplete for minimizing overall costs.

- ☐ Purchase an EC2 Instance Savings Plan. Optimize the Lambda functions' duration, memory usage, the number of invocations, and the amount of data that is transferred. Connect the Lambda functions to a public subnet in the same VPC where the EC2 instances run.

Similar to Option A, EC2 Instance Savings Plans don't cover Lambda costs. Additionally, placing Lambda functions in a public subnet, even in the same VPC, is less secure and may introduce unnecessary network hops compared to using the private subnet.

- ☒ Purchase a Compute Savings Plan. Optimize the Lambda functions' duration, memory usage, the number of invocations, and the amount of data that is transferred. Connect the Lambda functions to the private subnet that contains the EC2 instances.

A Compute Savings Plan provides savings on both EC2 and Lambda usage. Connecting Lambda functions to the private subnet with EC2 instances ensures low latency and a secure network configuration.

- ☐ Purchase a Compute Savings Plan. Optimize the Lambda functions' duration, memory usage, the number of invocations, and the amount of data that is transferred. Keep the

Lambda functions in the Lambda service VPC.

Keeping Lambda functions in the Lambda service VPC without direct network access to the EC2 instances will not fulfill the requirement of direct network access. While Compute Savings Plan is right, keeping the lambda in service VPC is incorrect.

Overall Explanation:

The correct solution is to purchase a Compute Savings Plan, optimize Lambda resource utilization, and connect the Lambda functions to the private subnet containing the EC2 instances. Compute Savings Plans offer savings on both EC2 and Lambda. Connecting the Lambda functions to the same private subnet minimizes latency and enhances security, and meets the direct network access requirement.

[View Discussions](#)[Ask AI](#)

Question 50

Not Attempted

Security



A solutions architect is tasked with granting a team access to Amazon S3 buckets residing in two distinct AWS accounts: a development account and a production account. Currently, team members access S3 buckets within the development account using individual IAM users assigned to an IAM group with appropriate permissions. In the production account, the architect has already created an IAM role with a policy granting access to an S3 bucket in that account. Which approach fulfills these requirements while adhering to the principle of least privilege?

- ☐ ☒ Attach the AdministratorAccess policy to the IAM users in the development account.
Attaching the AdministratorAccess policy grants excessive permissions, violating the principle of least privilege and potentially leading to security risks.
- ☒ Add the development account as a principal in the trust policy of the IAM role in the production account.
Adding the development account as a principal allows users from that account to assume the role in the production account, granting temporary access to the S3 bucket while adhering to the principle of least privilege.
- ☐ Disable the S3 Block Public Access feature on the S3 bucket in the production account.
Disabling S3 Block Public Access is not relevant to cross-account access and weakens security by potentially exposing the bucket to the public.

- ⊗ Create IAM users in the production account with unique credentials for each team member.

Creating individual IAM users in the production account complicates user management and scaling, making it less secure and violating the principle of least privilege.

Overall Explanation:

The optimal solution is to add the development account as a principal in the trust policy of the IAM role in the production account. This enables cross-account access via role assumption, ensuring team members gain temporary, least-privileged access to the production S3 bucket. Attaching AdministratorAccess (Option A) provides excessive permissions. Disabling Block Public Access (Option C) weakens security. Creating separate IAM users in production (Option D) complicates user management and does not adhere to the principle of least privilege.

 View Discussions

 Ask AI

Question 51

Not Attempted

Security



A company uses AWS Organizations with all features enabled and runs multiple Amazon EC2 workloads in the ap-southeast-2 Region. The company has a service control policy (SCP) that prevents any resources from being created in any other Region. A security policy requires the company to encrypt all data at rest. An audit discovers that employees have created Amazon Elastic Block Store (Amazon EBS) volumes for EC2 instances without encrypting the volumes. The company wants any new EC2 instances that any IAM user or root user launches in ap-southeast-2 to use encrypted EBS volumes. The company wants a solution that will have minimal effect on employees who create EBS volumes. Which combination of steps will meet these requirements?

- ⊗ In the Amazon EC2 console, select the EBS encryption account attribute and define a default encryption key.

Setting the default encryption key at the EC2 console level will enforce encryption within a single account but might not cover all accounts in the organization.

- ⊗ Create an IAM permission boundary. Attach the permission boundary to the root organizational unit (OU). Define the boundary to deny the ec2:CreateVolume action

when the `ec2:Encrypted` condition equals false.

IAM permission boundaries do not directly prevent actions across organizational units like SCPs do, and are more complex to implement for this scenario.

- ✔ Create an SCP. Attach the SCP to the root organizational unit (OU). Define the SCP to deny the `ec2:CreateVolume` action when the `ec2:Encrypted` condition equals false.

This option will prevent the creation of unencrypted EBS volumes at the organization level by denying the `ec2:CreateVolume` action when encryption is not enabled. SCPs are ideal for enforcing mandatory policies across all accounts.

- ✘ Update the IAM policies for each account to deny the `ec2:CreateVolume` action when the `ec2:Encrypted` condition equals false.

Updating IAM policies for each account is a more complex and less scalable solution than using SCPs. It would require managing policies across multiple accounts, which is not ideal.

- ✔ In the Organizations management account, specify the Default EBS volume encryption setting.

Setting the Default EBS volume encryption setting ensures that all new EBS volumes created within the organization are encrypted by default. This simplifies the process for employees and meets the encryption requirement.

Overall Explanation:

The best approach is to combine an SCP to strictly prevent the creation of unencrypted volumes (Option 3), with setting the Default EBS volume encryption at the organization level (Option 5). This ensures that all newly created volumes are encrypted by default, with the SCP acting as a hard stop to prevent non-compliant volumes, providing minimal disruption to employees' workflows .

 View Discussions

 Ask AI

Question 52

Not Attempted

Cost Optimization



A company operates a highly available image-processing application on Amazon EC2 instances within a single VPC, distributed across multiple subnets in different Availability Zones. The EC2 instances function independently, but they rely on downloading and uploading images from/to Amazon S3 via a single NAT gateway. The company is looking for the MOST cost-effective method to minimize data transfer charges. Which approach should they take?

⊗ Launch a NAT gateway in each Availability Zone.

Launching NAT gateways in each AZ can improve availability but doesn't avoid data transfer charges as traffic still traverses the NAT gateways.

⊗ Replace the NAT gateway with a NAT instance.

Replacing the NAT gateway with a NAT instance doesn't solve the data transfer cost issue, as data still needs to be transferred through the NAT instance.

✓ Deploy a gateway VPC endpoint for Amazon S3.

A gateway VPC endpoint allows access to S3 resources within the VPC without using the internet or a NAT gateway, thus avoiding regional data transfer charges.

⊗ Provision an EC2 Dedicated Host to run the EC2 instances.

Provisioning an EC2 Dedicated Host only affects the physical host and has no impact on data transfer charges related to S3 access.

Overall Explanation:

Deploying a gateway VPC endpoint for Amazon S3 is the most cost-effective solution. It enables direct connection between the VPC and S3, bypassing the NAT gateway and the public internet, thus preventing regional data transfer charges. The other options either do not address the core issue of data transfer costs related to S3 or are irrelevant to the problem.

 View Discussions

 Ask AI

Question 53

Not Attempted

Databases



A company aims to leverage an Amazon RDS for PostgreSQL DB cluster to streamline database administrative tasks for its production database workloads. The company requires high availability with automatic failover support in under 40 seconds in most scenarios. Additionally, they need to offload read operations from the primary instance while minimizing costs. Which solution meets these requirements?

⊗ Use an Amazon RDS Multi-AZ DB instance deployment. Create one read replica and direct read workload to the read replica.

Multi-AZ DB instance deployments typically have failover times exceeding 40 seconds (60-120 seconds), failing to meet the requirement. While read replicas can offload reads, the failover time is

a constraint.

- ⊗ Use an Amazon RDS Multi-AZ DB cluster deployment. Create two read replicas and direct the read workload to the read replicas.

While this option uses a Multi-AZ DB cluster which provides faster failover, creating additional read replicas increases costs. A DB cluster provides a reader endpoint by default.

- ⊗ Use an Amazon RDS Multi-AZ DB instance deployment. Direct the read workload to the secondary instance in the Multi-AZ pair.

Multi-AZ DB instance deployments do not allow direct read access to the secondary instance. Additionally, the failover times (60-120 seconds) are not suitable for the 40-second requirement.

- ✓ Use an Amazon RDS Multi-AZ DB cluster deployment. Direct the read workload to the reader endpoint.

Multi-AZ DB clusters provide automatic failover in under 40 seconds. The reader endpoint distributes read traffic across available reader instances, offloading the primary instance and helping keep costs as low as possible as no additional read replicas are required.

Overall Explanation:

Amazon RDS Multi-AZ DB clusters provide high availability with failover times of typically less than 35 seconds. They also allow read-only workloads to be directed to reader instances via the reader endpoint, thereby offloading the primary instance. This provides both high availability and read scalability while keeping costs low by utilizing the built-in reader endpoint without needing to create additional read replicas.

 View Discussions

 Ask AI

Question 54

Not Attempted

Serverless



A company operates a highly available SFTP service. Currently, the service utilizes two Amazon EC2 Linux instances, each with an Elastic IP address, to accept traffic from trusted IP sources over the internet. These instances rely on shared storage. User accounts are managed as Linux users on these SFTP servers. The company wants to migrate to a serverless solution for its SFTP service. This new solution must provide high IOPS performance, highly configurable security, and continued control over user permissions. Which solution best fits these requirements?

- ⊗ Create an encrypted Amazon Elastic Block Store (Amazon EBS) volume. Create an AWS Transfer Family SFTP service with a public endpoint that allows only trusted IP addresses. Attach the EBS volume to the SFTP service endpoint. Grant users access to the SFTP service.

This option is incorrect. AWS Transfer Family does not support EBS as a storage backend.

- ✔ Create an encrypted Amazon Elastic File System (Amazon EFS) volume. Create an AWS Transfer Family SFTP service with elastic IP addresses and a VPC endpoint that has internet-facing access. Attach a security group to the endpoint that allows only trusted IP addresses. Attach the EFS volume to the SFTP service endpoint. Grant users access to the SFTP service.

This is the best option. EFS provides a serverless, shared file system with high IOPS. AWS Transfer Family supports EFS. Using a VPC endpoint with an internet-facing access and a security group allows for restricting access to trusted IP addresses. EFS enables managing user permissions.

- ⊗ Create an Amazon S3 bucket with default encryption enabled. Create an AWS Transfer Family SFTP service with a public endpoint that allows only trusted IP addresses. Attach the S3 bucket to the SFTP service endpoint. Grant users access to the SFTP service.

While AWS Transfer Family supports S3, S3 doesn't inherently guarantee high IOPS performance comparable to EFS. Also configuring the public endpoint to only allow trusted IP addresses isn't directly achievable.

- ⊗ Create an Amazon S3 bucket with default encryption enabled. Create an AWS Transfer Family SFTP service with a VPC endpoint that has internal access in a private subnet. Attach a security group that allows only trusted IP addresses. Attach the S3 bucket to the SFTP service endpoint. Grant users access to the SFTP service.

This option is incorrect because the endpoint is internal to the private subnet, making it inaccessible from the public internet, as required for an SFTP service.

Overall Explanation:

The optimal solution is to use AWS Transfer Family with an Amazon EFS volume. EFS is serverless and offers high IOPS. AWS Transfer Family natively supports EFS. Using a VPC endpoint with elastic IP addresses that have internet-facing access, coupled with a security group that filters traffic by trusted IP addresses, satisfies the security requirements. EFS facilitates managing user permissions, fulfilling all the company's needs.

 View Discussions

 Ask AI

Question 55

Not Attempted

Serverless



A company is developing a new machine learning (ML) model solution on AWS. The models are developed as independent microservices that fetch approximately 1 GB of model data from Amazon S3 at startup and load the data into memory. Users access the models through an asynchronous API, sending requests (or batches of requests) and specifying where the results should be sent. The company provides models to hundreds of users, with irregular usage patterns. Some models might be unused for days or weeks, while others could receive batches of thousands of requests. Which architecture is the MOST suitable to meet these requirements efficiently and cost-effectively?

- ⊗ Direct the requests from the API to a Network Load Balancer (NLB). Deploy the models as AWS Lambda functions invoked by the NLB.

Lambda functions may not be suitable for large model sizes and long processing times due to limitations on memory and execution duration. Also, NLB might not be ideal for asynchronous processing.

- ⊗ Direct the requests from the API to an Application Load Balancer (ALB). Deploy the models as Amazon Elastic Container Service (Amazon ECS) services that read from an Amazon Simple Queue Service (Amazon SQS) queue. Use AWS App Mesh to scale the instances of the ECS cluster based on the SQS queue size.

Using ALB is unnecessary. App Mesh adds complexity that is not required, the ECS scaling isn't directly tied to the SQS Queue, making it not ideal.

- ⊗ Direct the requests from the API into an Amazon Simple Queue Service (Amazon SQS) queue. Deploy the models as AWS Lambda functions that are invoked by SQS events. Use AWS Auto Scaling to increase the number of vCPUs for the Lambda functions based on the SQS queue size.

Lambda functions may not be suitable for large model sizes and long processing times due to limitations on memory and execution duration. Lambda doesn't directly scale vCPUs.

- ✔ Direct the requests from the API into an Amazon Simple Queue Service (Amazon SQS) queue. Deploy the models as Amazon Elastic Container Service (Amazon ECS) services that read from the queue. Enable AWS Auto Scaling on Amazon ECS for both the cluster and copies of the service based on the queue size.

This design is optimal. SQS decouples the API from the model processing. ECS provides a containerized environment suitable for running ML models. Auto Scaling adjusts the ECS cluster and service capacity based on the queue size, ensuring efficient scaling and cost optimization.

Overall Explanation:

The best solution involves using Amazon SQS to decouple the API from the model processing microservices, which are deployed as Amazon ECS services. SQS handles the asynchronous requests. ECS provides a suitable environment for the ML models, and AWS Auto Scaling dynamically adjusts the ECS cluster and service based on the SQS queue size, ensuring efficient resource utilization and cost optimization. Lambda is not suitable for large models due to memory constraints and execution time limitations.

[View Discussions](#)[Ask AI](#)

Question 56

Not Attempted

Security



A solutions architect wants to use the following JSON text as an identity-based policy to grant specific permissions:

```
{
  "Statement": [
    {
      "Action": [
        "ssm:ListDocuments",
        "ssm:GetDocument"
      ],
      "Effect": "Allow",
      "Resource": "*",
      "Sid": ""
    }
  ],
  "Version": "2012-10-17"
}
```

examtopics.io

Which IAM entities can the solutions architect attach this policy to? (Choose two.)

☒ Role

IAM roles can be assigned identity-based policies to define their permissions.

**Group**

IAM groups can be assigned identity-based policies to manage permissions for multiple users.

**Organization**

Organizations use Service Control Policies (SCPs) for permission management, not identity-based policies directly.

**Amazon Elastic Container Service (Amazon ECS) resource**

ECS resources utilize IAM roles for permissions, not direct attachment of identity-based policies.

**Amazon EC2 resource**

EC2 resources utilize IAM roles for permissions, not direct attachment of identity-based policies.

Overall Explanation:

Identity-based policies in IAM can be attached to IAM roles and IAM groups. These policies define the permissions granted to the role or group. Organizations use SCPs and ECS/EC2 resources use IAM roles. Therefore, the correct answers are Role and Group.

[View Discussions](#)[Ask AI](#)

Question 57

Not Attempted

[Cost Optimization](#)

A company is running a custom application on Amazon EC2 instances. The application has frontend nodes that require continuous operation (24/7) and backend nodes that are used for batch processing, with the number of required backend nodes fluctuating throughout the day based on workload demands. To optimize costs while ensuring consistent performance for the frontend and scalability for the backend, which EC2 instance purchasing options should the company implement?

**Use Reserved Instances for the frontend nodes. Use AWS Fargate for the backend nodes.**

AWS Fargate is designed for containerized applications, and the question does not specify that the application is containerized. Also, Fargate might not be the most cost-effective solution for batch processing with fluctuating workloads.

- ☒ Use Reserved Instances for the frontend nodes. Use Spot Instances for the backend nodes.

This is the most cost-effective solution. Reserved Instances are ideal for the consistently running frontend nodes, providing significant cost savings. Spot Instances are suitable for the flexible backend nodes, as they can be provisioned and terminated based on workload, and are cheaper than on-demand instances.

- ☐ Use Spot Instances for the frontend nodes. Use Reserved Instances for the backend nodes.

Spot Instances are not suitable for the frontend nodes because their operation is not guaranteed due to potential interruptions, and the frontend nodes need to be running 24/7. Reserved instances are for the backend which runs for a short period of time.

- ☐ Use Spot Instances for the frontend nodes. Use AWS Fargate for the backend nodes.

Spot Instances are not a reliable choice for the 24/7 frontend nodes. Also, Fargate may not be appropriate since the app is not containerized.

Overall Explanation:

The most cost-effective solution involves using Reserved Instances for the consistently running frontend nodes and Spot Instances for the dynamically scaling backend nodes. Reserved Instances provide cost savings for predictable workloads, while Spot Instances offer significant discounts for workloads that can tolerate interruptions. Fargate is designed for containers, so Reserved + Spot instances are ideal.

 View Discussions

 Ask AI

Question 58

Not Attempted

Storage



A company currently runs its workloads on-premises, utilizing high block storage capacity. Their daily peak input/output transactions per second (IOPS) do not exceed 15,000. The company aims to migrate these workloads to Amazon EC2 and requires the ability to provision disk performance independently of storage capacity. Which Amazon Elastic Block Store (Amazon EBS) volume type will meet these requirements MOST cost-effectively?

- ☐ GP2 volume type

GP2 volumes offer a balance of price and performance but don't allow independent provisioning of IOPS and storage capacity as effectively as other options, potentially leading to higher costs if

storage capacity is oversized to meet IOPS needs.

⊗ io2 volume type

io2 volumes are designed for high-performance, I/O intensive applications and allow for independent provisioning of IOPS and storage. While suitable, they are typically more expensive than GP3 for workloads with moderate IOPS requirements such as this scenario.

✓ GP3 volume type

GP3 volumes offer a cost-effective balance of price and performance, allowing independent provisioning of IOPS and storage capacity. This is ideal for workloads where IOPS requirements are known and can be provisioned separately from storage size, ensuring optimal cost efficiency.

⊗ io1 volume type

io1 volumes are also designed for high-performance I/O intensive applications, similar to io2, and enable independent provisioning of IOPS and storage. However, like io2, they are generally more expensive than GP3 for workloads with relatively moderate IOPS requirements.

Overall Explanation:

GP3 volumes provide the most cost-effective solution for provisioning disk performance independently of storage capacity while meeting the 15,000 IOPS requirement. GP2 doesn't offer independent provisioning as effectively, and io1 and io2 are more expensive options designed for higher performance workloads than this scenario requires. GP3 allows up to 16000 IOPS and costs less than GP2 per GB.

 View Discussions

 Ask AI

Question 59

Not Attempted

Storage



A company's healthcare application, hosted on an on-premises infrastructure facing storage limitations, requires a secure migration to AWS. Due to a new regulation, the company must implement audit access at all levels of the stored data, and the application data changes frequently. Which solution satisfies these requirements?

✓ Use AWS DataSync to move the existing data to Amazon S3. Use AWS CloudTrail to log data events.

AWS DataSync efficiently migrates data, especially when it changes frequently. Logging data events with CloudTrail provides comprehensive audit access at all levels of the stored data in S3.

- ☒ Use AWS Snowcone to move the existing data to Amazon S3. Use AWS CloudTrail to log management events.
AWS Snowcone is suitable for edge computing and data transfer in rugged environments. Logging management events only captures administrative actions, not the required data access auditing.
- ☒ Use Amazon S3 Transfer Acceleration to move the existing data to Amazon S3. Use AWS CloudTrail to log data events.
S3 Transfer Acceleration improves transfer speeds, but it doesn't provide the data migration capabilities of DataSync. While CloudTrail logging of data events is correct, the migration method is not the most suitable.
- ☒ Use AWS Storage Gateway to move the existing data to Amazon S3. Use AWS CloudTrail to log management events.
AWS Storage Gateway is primarily for hybrid cloud storage, connecting on-premises applications to AWS storage. Logging management events does not meet the data access auditing requirement.

Overall Explanation:

AWS DataSync offers a method for continuous data synchronization. Combined with AWS CloudTrail logging of data events, this approach ensures the data is migrated to S3 and that all access to data is audited for compliance. Storage Gateway creates a hybrid solution which is not what is asked for. Management events do not track data access.

[View Discussions](#)[Ask AI](#)

Question 60

Not Attempted

[Compute](#)

A solutions architect is implementing a complex Java application with a MySQL database. The Java application must be deployed on Apache Tomcat and must be highly available. Which of the following approaches would BEST meet these requirements?

- ☒ Deploy the application in AWS Lambda. Configure an Amazon API Gateway API to connect with the Lambda functions.
AWS Lambda is a serverless compute service that runs code without provisioning or managing servers. While suitable for many applications, it might not be the best fit for a complex, long-running Java application designed for Apache Tomcat.

- ✓ Deploy the application by using AWS Elastic Beanstalk. Configure a load-balanced environment and a rolling deployment policy.

AWS Elastic Beanstalk is an easy-to-use service for deploying and scaling web applications and services developed with Java, PHP, Node.js, Python, Ruby, Go, and Docker on familiar servers such as Apache, Nginx, Passenger, and IIS. A load-balanced environment provides high availability, and a rolling deployment policy minimizes downtime during updates.

- ✗ Migrate the database to Amazon ElastiCache. Configure the ElastiCache security group to allow access from the application.

Amazon ElastiCache is an in-memory data store and cache service. It is not suitable for replacing a persistent database like MySQL. Furthermore, the question specifies requirements for the application deployment, not database migration.

- ✗ Launch an Amazon EC2 instance. Install a MySQL server on the EC2 instance. Configure the application on the server. Create an AMI. Use the AMI to create a launch template with an Auto Scaling group.

While this approach can achieve high availability, it involves significantly more manual configuration and management compared to using a managed service like Elastic Beanstalk. It also increases operational overhead.

Overall Explanation:

AWS Elastic Beanstalk provides a platform as a service (PaaS) that simplifies the deployment and management of Java applications on Apache Tomcat. By configuring a load-balanced environment and a rolling deployment policy, the solutions architect can achieve both high availability and minimal downtime during updates. This approach minimizes the operational overhead compared to manually managing EC2 instances and Auto Scaling groups.

 View Discussions

 Ask AI

Question 61

Not Attempted

Security



A serverless application, leveraging Amazon API Gateway, AWS Lambda, and Amazon DynamoDB, requires the Lambda function to possess the necessary permissions for reading from and writing to a specific DynamoDB table. Which solution offers the MOST secure approach to grant the Lambda function this access?

- ⊗ Create an IAM user with programmatic access for the Lambda function. Attach a policy to this user granting read/write access to the DynamoDB table. Store the ``access_key_id`` and ``secret_access_key`` as environment variables within the Lambda function configuration. Restrict other AWS users from modifying the Lambda function configuration.

Storing credentials directly in environment variables or using an IAM user for Lambda functions is generally discouraged due to security risks. It makes credential rotation difficult and increases the risk of exposure.

- ✔ Create an IAM role that trusts Lambda as a service. Attach a policy to the role that allows read/write access to the DynamoDB table. Configure the Lambda function to assume this role as its execution role.

This is the recommended approach. IAM roles provide temporary credentials and are managed by AWS. By associating a role with the Lambda function, you grant it permissions without embedding long-term credentials.

- ⊗ Create an IAM user with programmatic access for the Lambda function. Attach a policy granting read/write access to the DynamoDB table. Store the ``access_key_id`` and ``secret_access_key`` securely as secure string parameters in AWS Systems Manager Parameter Store. Modify the Lambda function code to retrieve these parameters before accessing the DynamoDB table.

While using Systems Manager Parameter Store is better than storing credentials directly, it still relies on long-term credentials for the Lambda function, which can be vulnerable if compromised. IAM roles are a more secure alternative.

- ⊗ Create an IAM role that trusts DynamoDB as a service. Attach a policy to the role allowing access from the Lambda function. Update the Lambda function code to assume this new role as its execution role.

This option is incorrect because DynamoDB does not need to trust Lambda. It is the Lambda function that requires permissions to access DynamoDB. The IAM role needs to trust Lambda as a service so Lambda can assume this role.

Overall Explanation:

The most secure method is to utilize an IAM role that trusts Lambda as a service and grants the necessary DynamoDB permissions. This avoids storing long-term credentials within the Lambda function itself. Instead, the Lambda function assumes the role, receiving temporary credentials from the IAM service.

 View Discussions

 Ask AI

Question 62

Not Attempted

Security



An IAM policy is attached to an IAM group as shown below. This is the only policy applied to the group.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "1",
      "Effect": "Allow",
      "Action": "ec2:*",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "ec2:Region": "us-east-1"
        }
      }
    },
    {
      "Sid": "2",
      "Effect": "Deny",
      "Action": [
        "ec2:StopInstances",
        "ec2:TerminateInstances"
      ],
      "Resource": "*",
      "Condition": {
        "BoolIfExists": {"aws:MultiFactorAuthPresent": false}
      }
    }
  ]
}
```

examtopics.io

What are the effective IAM permissions for members of this group?

- ☐ Group members are permitted any Amazon EC2 action within the us-east-1 Region. Statements after the Allow permission are not applied.

This option is incorrect because the policy includes a Deny statement that modifies the permissions granted by the Allow statement.

- ☐ Group members are denied any Amazon EC2 permissions in the us-east-1 Region unless they are logged in with multi-factor authentication (MFA).

This option is incorrect because the Deny statement only applies to ec2:StopInstances and ec2:TerminateInstances actions, not all EC2 actions.

- ☐ Group members are allowed the `ec2:StopInstances` and `ec2:TerminateInstances` permissions for all Regions when logged in with multi-factor authentication (MFA). Group members are permitted any other Amazon EC2 action.

This option is incorrect because the Deny statement is scoped to the us-east-1 region.

- ☒ Group members are allowed the `ec2:StopInstances` and `ec2:TerminateInstances` permissions for the us-east-1 Region only when logged in with multi-factor authentication (MFA). Group members are permitted any other Amazon EC2 action within the us-east-1 Region.

This option correctly describes the effective permissions. The Allow statement grants all EC2 actions in us-east-1, but the Deny statement restricts the ability to stop or terminate instances in us-east-1 unless MFA is used.

Overall Explanation:

The policy allows all EC2 actions in the us-east-1 region. However, a Deny statement restricts the ability to perform the `ec2:StopInstances` and `ec2:TerminateInstances` actions within the us-east-1 region, unless the user is authenticated with MFA. Therefore, option D is the correct answer.

 View Discussions

 Ask AI

Question 63

Not Attempted

Networking



A company's on-premises application generates a large amount of time-sensitive data that is backed up to Amazon S3. The application's growth has led to user complaints about internet bandwidth limitations. A solutions architect needs to design a long-term solution that allows for both timely backups to Amazon S3 with minimal impact on internet connectivity for internal users. Which solution meets these requirements?

- ☐ Establish AWS VPN connections and proxy all traffic through a VPC gateway endpoint. VPN connections still utilize internet bandwidth and may not effectively address the issue of limited bandwidth.
- ☒ Establish a new AWS Direct Connect connection and direct backup traffic through this new connection.

AWS Direct Connect provides a dedicated, private network connection between on-premises and AWS, bypassing the public internet. This ensures fast, reliable transfers without impacting internet

connectivity for internal users.

- ⊗ Order daily AWS Snowball devices. Load the data onto the Snowball devices and return the devices to AWS each day.

Daily Snowball transfers are not a practical long-term solution for frequent backups due to cost and operational overhead. Also, it won't be timely.

- ⊗ Submit a support ticket through the AWS Management Console. Request the removal of S3 service limits from the account.

S3 service limits do not address the issue of internet bandwidth limitations.

Overall Explanation:

AWS Direct Connect is the most suitable long-term solution. It establishes a dedicated network connection between the on-premises data center and AWS, bypassing the public internet and providing more reliable, lower-latency communication for timely backups while minimizing the impact on internet bandwidth for internal users. VPN still relies on the internet, Snowball is not suited for daily time sensitive backups, and S3 limits are not related to the problem.

[View Discussions](#)[Ask AI](#)

Question 64

Not Attempted

Storage



A manufacturing company uses machine sensors that upload .csv files to an Amazon S3 bucket. These .csv files must be converted into images and made available as soon as possible for the automatic generation of graphical reports. The images become irrelevant after 1 month. However, the .csv files must be retained for training machine learning (ML) models twice a year. The ML training and audit schedules are determined weeks in advance. Which combination of actions will meet these requirements MOST cost-effectively? (Choose two.)

- ⊗ Launch an Amazon EC2 Spot Instance that downloads the .csv files every hour, generates the image files, and uploads the images to the S3 bucket.

Using EC2 is not cost-effective as it is always running or scheduled. Also, the images must be generated as soon as possible, meaning that waiting every hour to execute the image generation does not satisfy the business requirements.

- ✓ Design an AWS Lambda function that converts the .csv files into images and stores the images in the S3 bucket. Invoke the Lambda function when a .csv file is uploaded.
- Using Lambda is serverless and cost-effective as it is only running when a new .csv is uploaded. It also satisfies the business requirements of creating images as soon as possible.
- ✓ Create S3 Lifecycle rules for .csv files and image files in the S3 bucket. Transition the .csv files from S3 Standard to S3 Glacier 1 day after they are uploaded. Expire the image files after 30 days.
- Glacier is the cheapest storage class to store the .csv files. As the ML training and audit schedules are determined weeks in advance, the .csv files can be stored to Glacier. The image files can be expired after 30 days to save cost.
- ✗ Create S3 Lifecycle rules for .csv files and image files in the S3 bucket. Transition the .csv files from S3 Standard to S3 One Zone-Infrequent Access (S3 One Zone-IA) 1 day after they are uploaded. Expire the image files after 30 days.
- S3 One Zone-IA is more expensive than Glacier. Also, there is a minimum storage duration of 30 days. As the ML training and audit schedules are determined weeks in advance, the .csv files can be stored to Glacier.
- ✗ Create S3 Lifecycle rules for .csv files and image files in the S3 bucket. Transition the .csv files from S3 Standard to S3 Standard-Infrequent Access (S3 Standard-IA) 1 day after they are uploaded. Keep the image files in Reduced Redundancy Storage (RRS).
- S3 Standard-IA is more expensive than Glacier. Also, using RRS is not a cost-effective as image files only need to be available for 30 days.

Overall Explanation:

The most cost-effective solution is to use Lambda to generate the images as soon as possible when a .csv file is uploaded to S3. Also, Glacier is the cheapest storage class to store .csv files and satisfies the business requirements because the machine learning training and audits are planned weeks in advance. Therefore, options B and C are the correct answers.

[View Discussions](#)[Ask AI](#)

Question 65

Not Attempted

[Databases](#)

A company has developed a new video game as a web application. The application is in a three-tier architecture in a VPC with Amazon RDS for MySQL in the database layer. Several players will compete concurrently online. The game's developers want to display a top-10 scoreboard in near-real time and offer the ability to stop and restore the game while preserving the current scores. What should a solutions architect do to meet these requirements?

- ⊗ Set up an Amazon ElastiCache for Memcached cluster to cache the scores for the web application to display.

Memcached is a distributed memory object caching system and primarily used for caching arbitrary data (strings, objects) from results of database calls, API calls, or page rendering. It doesn't support complex operations such as sorting and ranking which are required to create a leaderboard.

- ✓ Set up an Amazon ElastiCache for Redis cluster to compute and cache the scores for the web application to display.

Redis provides fast in-memory data storage and processing and supports sorted sets, which are ideal for implementing leaderboards. Redis also supports persistence, allowing scores to be preserved across restarts.

- ⊗ Place an Amazon CloudFront distribution in front of the web application to cache the scoreboard in a section of the application.

CloudFront is a content delivery network (CDN) service that caches static and dynamic web content. It's not suitable for computing or storing frequently changing, real-time data like game scores.

- ⊗ Create a read replica on Amazon RDS for MySQL to run queries to compute the scoreboard and serve the read traffic to the web application.

While a read replica can offload read traffic from the primary database, querying the database for every scoreboard update would still introduce latency and potential performance bottlenecks. It doesn't provide the near-real-time performance required.

Overall Explanation:

ElastiCache for Redis is the most suitable choice. Redis offers in-memory data storage with very low latency and supports data structures like sorted sets, which are perfect for implementing real-time leaderboards. Furthermore, Redis offers persistence options to ensure data isn't lost on restarts or failures, meeting the requirement to preserve scores.

 View Discussions

 Ask AI

[← Back to Exam](#)



Marketing

[Affiliate Program](#)

[Unlimited Access](#)

[Business Unlimited Access](#)

Support

[Contact Us](#)

team@itcertifications.io

Company

[Udemy Alternative](#)

[Whizlabs Alternative](#)

[TutorialsDojo Alternative](#)

Legal

[Terms of service](#)

[Privacy policy](#)

[Refund policy](#)

[Shipping policy](#)



© 2026, ITcertifications Technologies. All rights reserved.